Москва 07.10.2026



# Как создать 400 пайплайнов загрузки данных за 1 месяц

Павел Мельников
Руководитель ИТ-проектов, Axenix

# Правила сказки

**Это история одного внедрения**

- Всё, что вы услышите, происходило на самом деле. Почти.
- Каждый персонаж = отдел = типовой заказчик данных
- Акционер. Аналитик. Разработчик. Четыре отдела.



# Информация о заказчике

## АО «ПоленоКлючСнаб» — снабжение Буратин золотыми ключиками собственного производства

ОРГСХЕМА

| Отдел | Сотрудники/Локации |
|---|---|
| Акционер | Тортилла |
| Директор | Карабас-Барабас |
| Финансы | Дуремар |
| Продажи | лягушки (Большой Пруд, Тихий Омут, Камышовый) |
| Маркетинг | лиса Алиса |
| Контроль качества (ОТК) | кот Базилио |
| Аналитика | Арлекин |
| Разработка | папа Карло + девопёс Артемон |

SKU

| Продукт | Цена, сольдо с НДС |
|---|---|
| Ключик «Стандарт» (латунь под золото) | 12 сол. |
| Ключик «Премиум» (сусальное золото) | 35 сол. |
| Ключик «Коллекционный» (гравировка Тортиллы) | 120 сол. |

Каналы продаж

| Канал | Тип | Покупатели |
|---|---|---|
| Точки у прудов | розница | Буратины |
| «ПрудМаркет» | маркетплейс | Буратины |
| Прямые контракты | опт | Кукольные театры («Огниво», «Театр Марионеток») |

# Источники данных



Все отчеты собираются из разных систем.

| Система | Тип | Данные | Владелец |
|---|---|---|---|
| duremar_finance | PostgreSQL | проводки, план-факт | Дуремар |
| pond_sales | PostgreSQL | заказы трех прудов | лягушки |
| basilio_qc | PostgreSQL | брак по партиям | кот Базилио |
| alisa_ads_reports | S3 | выгрузки рекламных площадок | лиса Алиса |

Склейка происходит вручную. В excel. По ночам.

# Проблематика: «Всё через папу Карло»

Ресурса папы Карло не хватает для оперативного решения рабочих задач.

- Всё через папу Карло
- 4 отдела — 1 разработчик
- Каждая заявка: скрипт + cron + файл по FTP
- Очередь — на три недели вперёд

«Ваша заявка сорок седьмая в очереди. Может быть, к пятнице.» — папа Карло

# Проблематика: «Испорченный телефон» требований

Смыслы теряются – бизнес-процессы простаивают.

ПРИМЕР

| **Кто** | **Что сказал** |
|---|---|
| Дуремар | «Дай отчёт по марже по прудам за сентябрь» |
| Арлекин | «Выгрузка: продажи минус закупки, группировка по точкам, 01.09–30.09» |
| папа Карло | orders o JOIN purchases p on o.pond_id = p.point_id WHERE dt BETWEEN '09-01' AND '09-30' |
| Дуремар *(через 3 недели)* | «Это не та разбивка. И где Камышовый?» |

«Я черепаха. Но даже с моей точки зрения вы все делаете очень медленно.» - акционер Тортилла

# Решение – Dat.ax



# Dat.ax – Подготовка данных

**Инструмент для работы с данными в режиме самообслуживания**
Позволяет консультантам / аналитикам (бизнес-пользователям) в режиме **Low-code** и **Self Service** (самообслуживания) загружать и объединять данные из **различных источников**, настраивать правила **преобразования и проверки данных**, сохранять результаты обработки данных в БД и файлах для **последующего анализа**.

Инструмент будет полезен для различных групп пользователей, в том числе пользователей без глубоких технических навыков, и позволит:

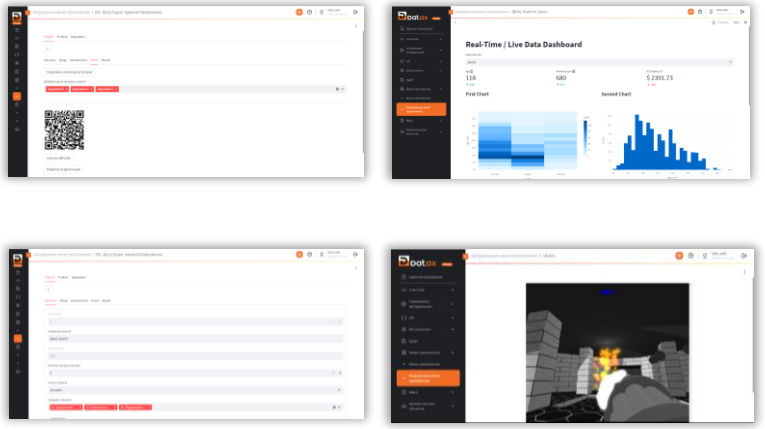

Оперативно подготавливать для анализа данные из различных источников


Получить прозрачный, задокументированный процесс подготовки данных


Повторно использовать технические решения для подготовки данных без программирования


Автоматизировать регулярные процессы подготовки витрин данных для любых пользовательских задач

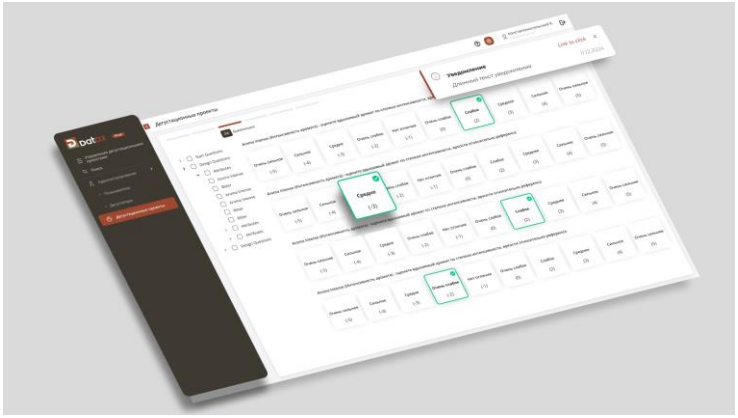
# Набор инструментов для работы с данными




## Мини-приложения

Встроенная в Dat.ax возможность создания аналитических приложений на языке Python с использованием собственной библиотеки для быстрого доступа к функциональности:

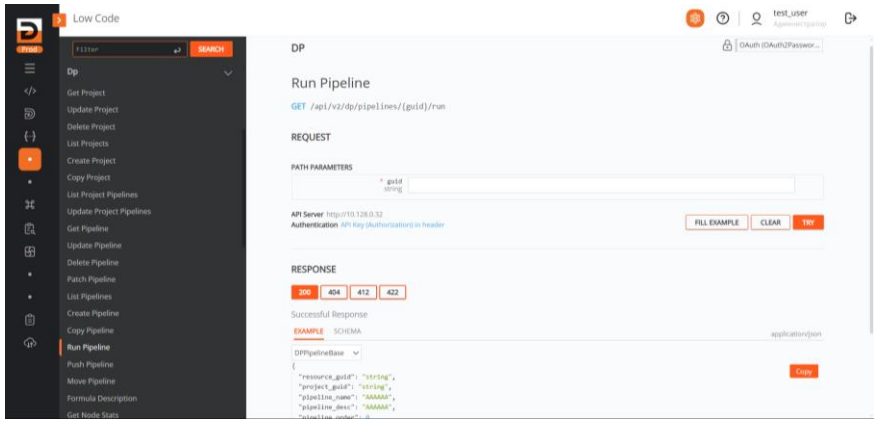
- Безопасность
- Работа с подключениями, процессами и др. мета-объектами
- Реализация любой необходимой дополнительного логики



## Custom’ные модули

Разработка под любые сценарии и требования, ускоренная за счёт переиспользования уже реализованных в платформе стандартных компонентов:

- Возможности создания нового модуля
- Табличные формы и элементы управления
- Функции управления доступам и ролями




## Открытый API

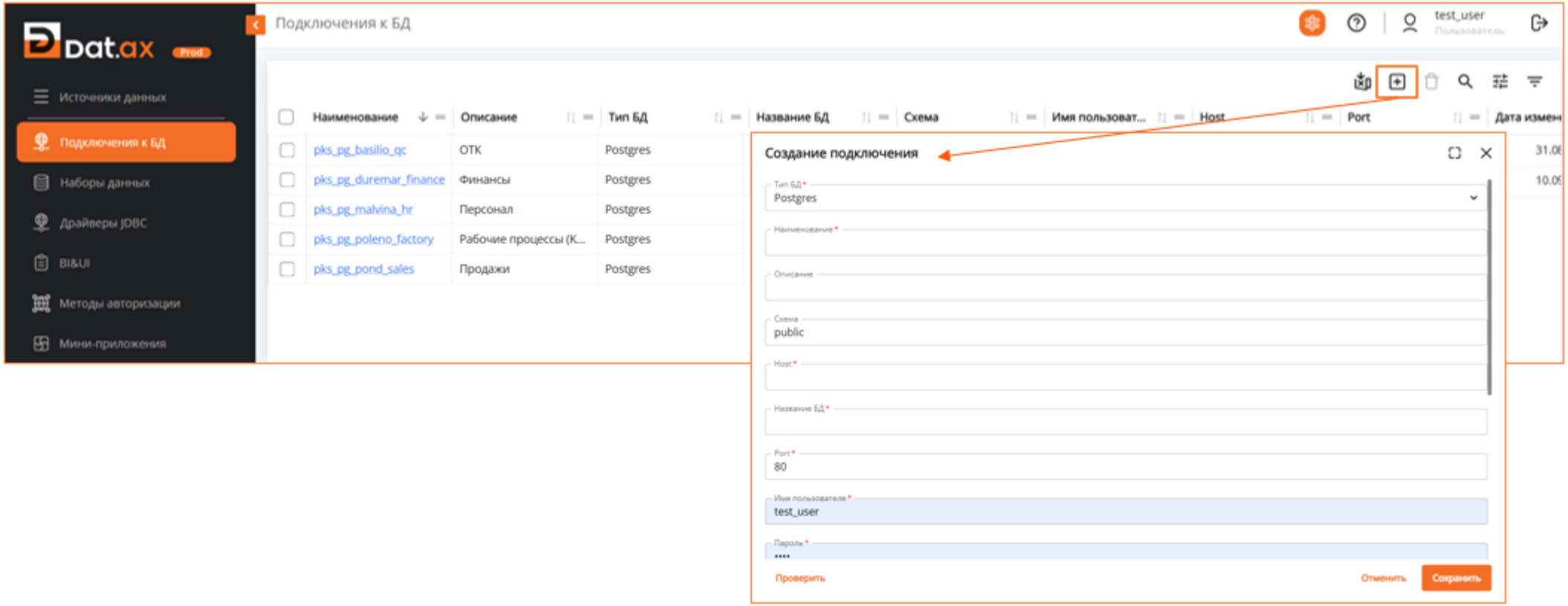
Документированный открытый REST API для интеграции с Dat.ax позволяет интегрировать решения в любой программно-аппаратный комплекс, информационную системы или заказное уникальное решение

# Создание подключения

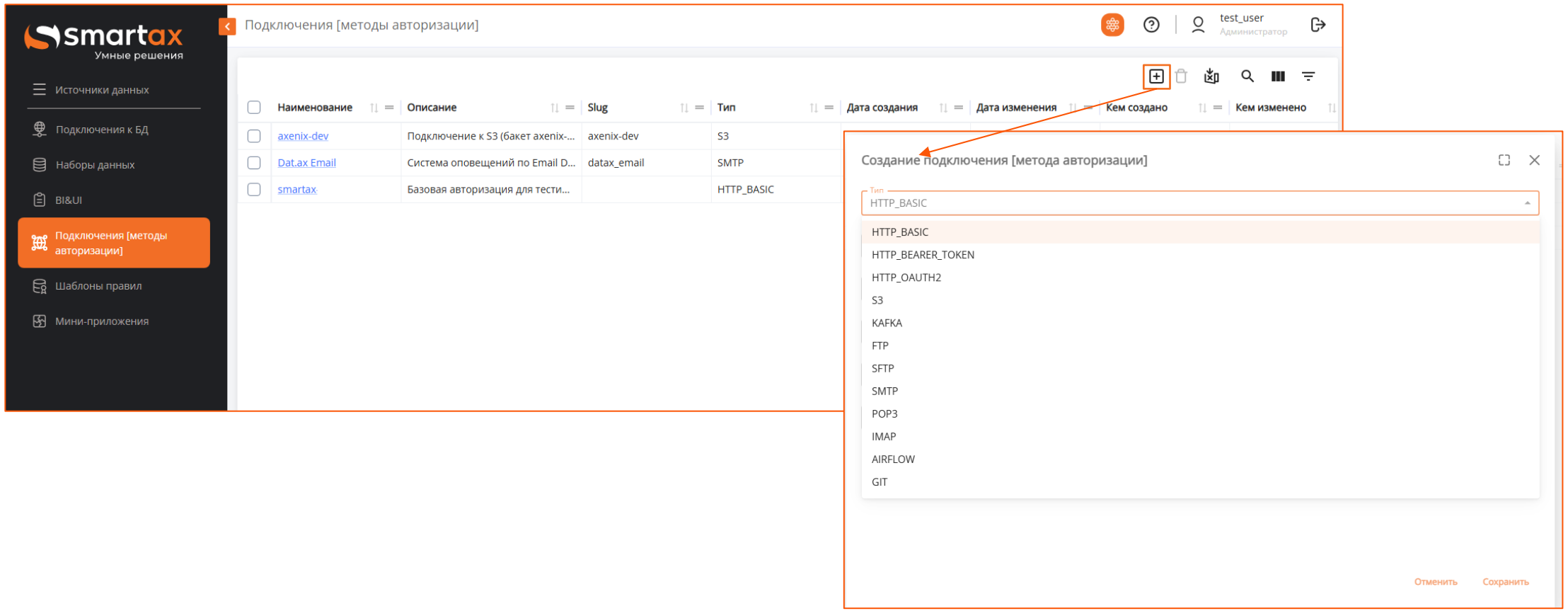


# Подключения к БД

**Подключение** хранит метаданные для соединения с Базой данных. Доступные подключения можно увидеть в навигационной панели – Модуль **Источники данных** – раздел **Подключения к БД**. Здесь можно просматривать существующие подключения, а также создавать, изменять и удалять их.
Для создания подключения нажмите **Создать**, заполните параметры БД и реквизиты доступа. Перед сохранением можно проверить доступность БД по указанным параметрам с помощью кнопки **Проверить**.



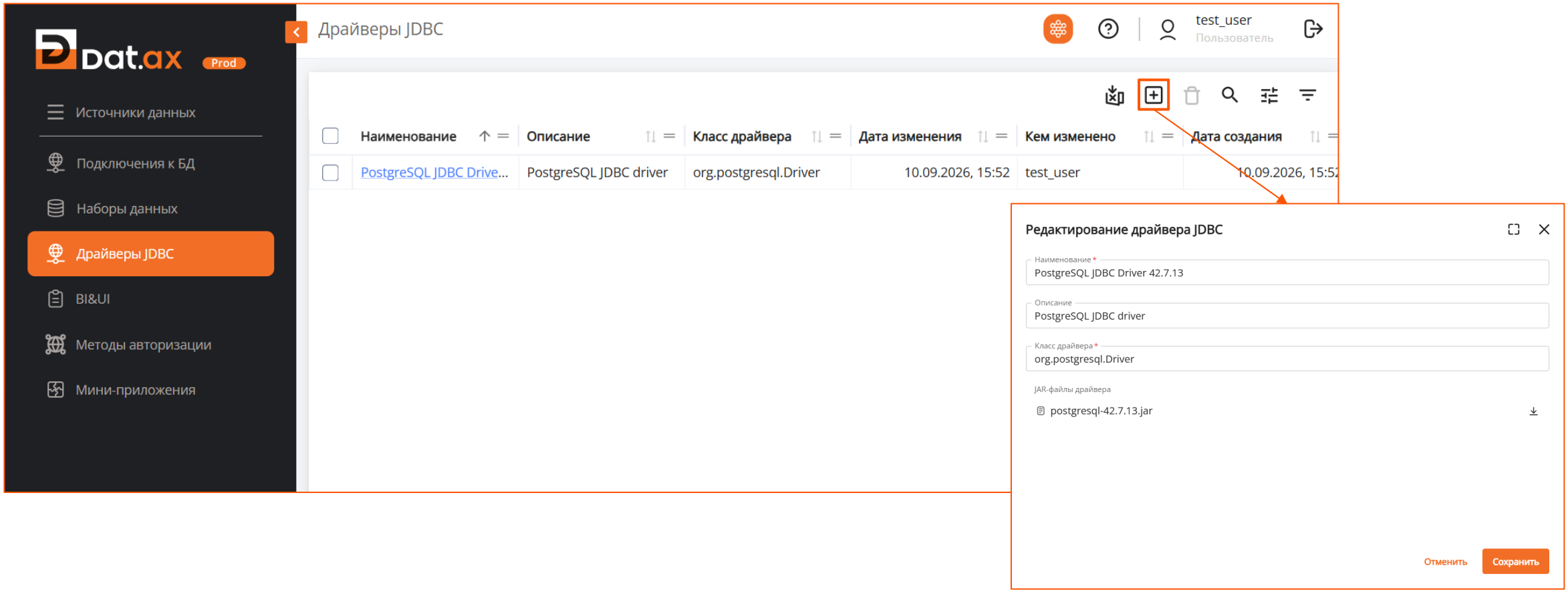
# Методы авторизации




**Источники данных. Подключения [методы авторизации]**

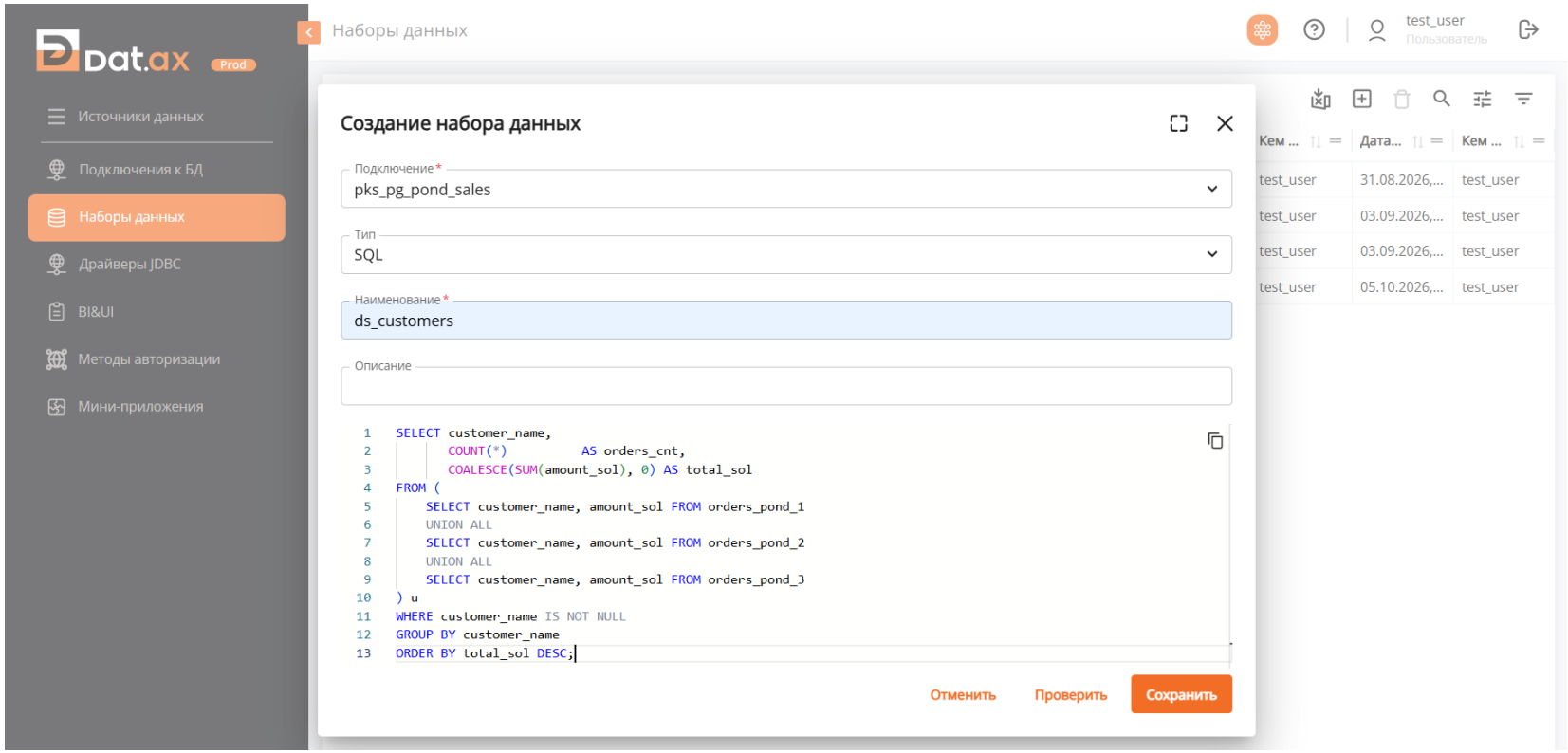
# Драйверы JDBC

При помощи подраздела **Драйверы JDBC** пользователь может настраивать подключения к реляционным базам данных с использованием JDBC драйверов. Сперва в разделе «Драйверы JDBC» создается драйвер, затем в разделе «Подключения к БД» создается подключение с использованием только что созданного драйвера.



# Наборы данных

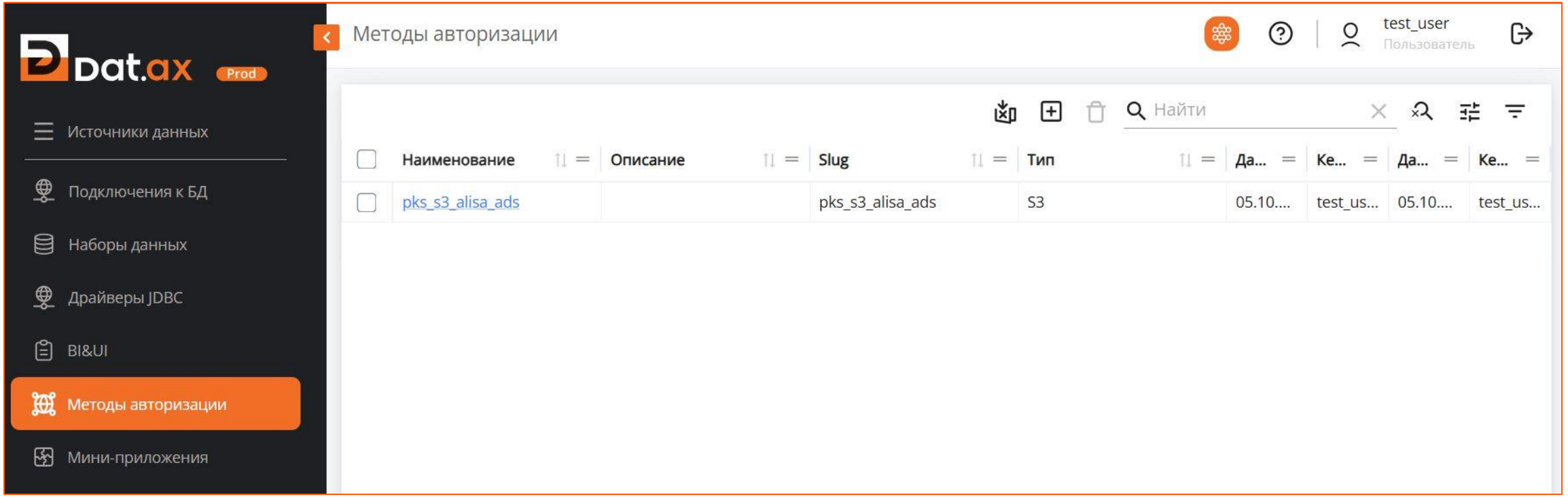
Арлекин решил, что будет удобно иметь справочник клиентов. Для этого он в разделе «источники данных» формирует объединенный набор данных с клиентами с использованием SQL запроса.
Теперь из любого пайплайна к нему можно обращаться как к обычной view.




**Подраздел Наборы данных**

# Получение данных из файлов

К папе Карло пришла Алиса и попросила дать ей инструмент добавления в пайплайны выгрузок с рекламных площадок. Для этого Папа Карло в методах авторизации настраивает подключение к бакету S3.



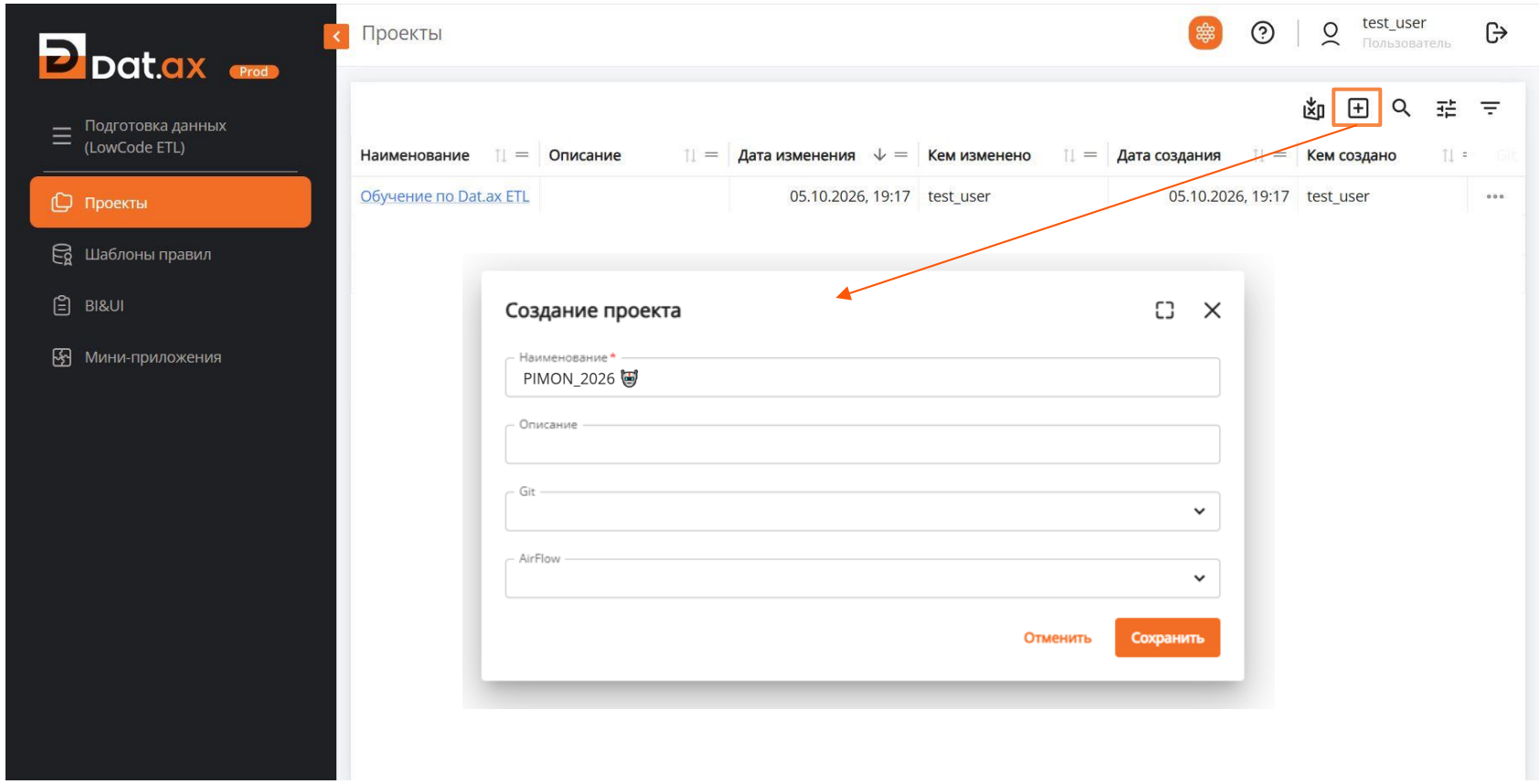

Теперь Алиса может добавлять файлы в хранилище и работать с ними в интерфейсе Dat.ax

# Проекты и пайплайны в Dat.ax



# Теория: Проект

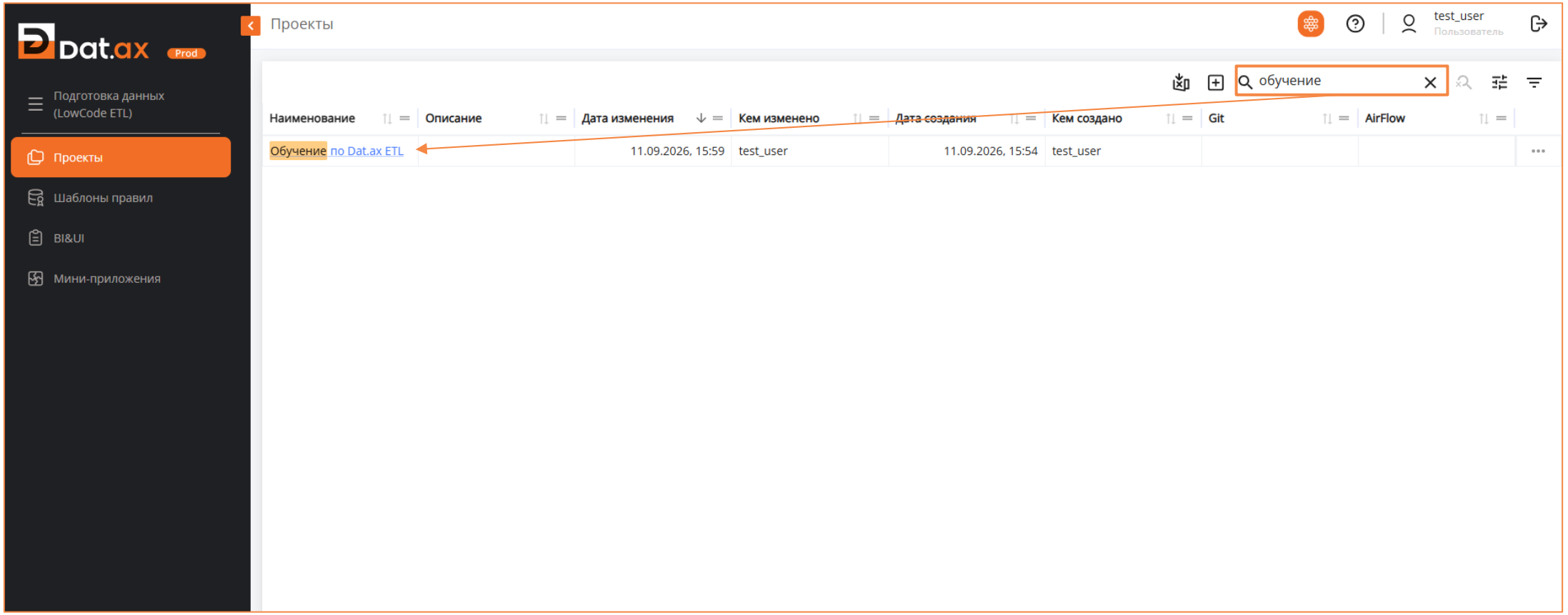
**Проект** представляет собой набор пайплайнов для работы с данными. Создание проекта выполняется в разделе **Проекты** с помощью кнопки **Создать**. В открывшейся форме указываются наименование и описание проекта.




**Создание проекта**

# Теория: Работа с пайплайном

Для перехода к работе с пайплайнами необходимо открыть нужный проект в разделе **Проекты**. Проект можно найти в общем списке или с помощью строки поиска. После выбора проекта открывается его рабочая область с доступными пайплайнами.




**Открытие проекта**

# Теория: Работа с пайплайном

Внутри проекта может быть неограниченное количество пайплайнов. При создании проекта автоматически создается первый пайплайн с идентичным проекту наименованием.

Новый пайплайн создаётся в текущем проекте с помощью кнопки **Создание пайплайна** + в меню управления пайплайном. При создании необходимо указать его наименование и выбрать подключение для stage. Выбранное подключение задаёт **Stage-подключение** пайплайна, в котором сохраняются промежуточные результаты выполнения узлов.

Поля описание и dag_id являются необязательными для заполнения.

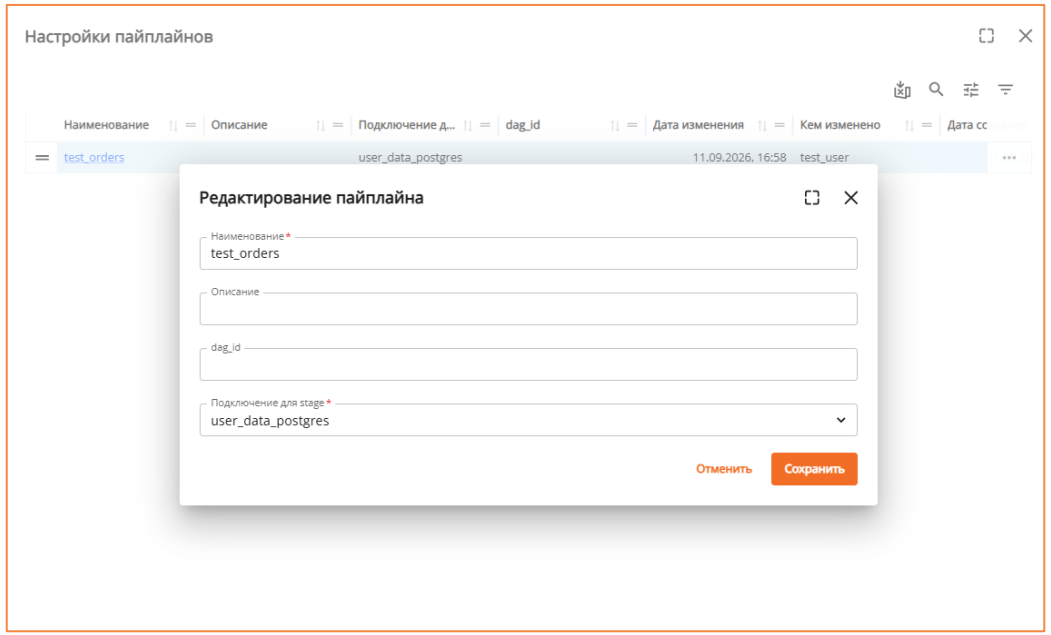



**Создание пайплайна**

# Теория: Работа с пайплайном

Если необходимо редактировать/переместить/копировать пайплайн:

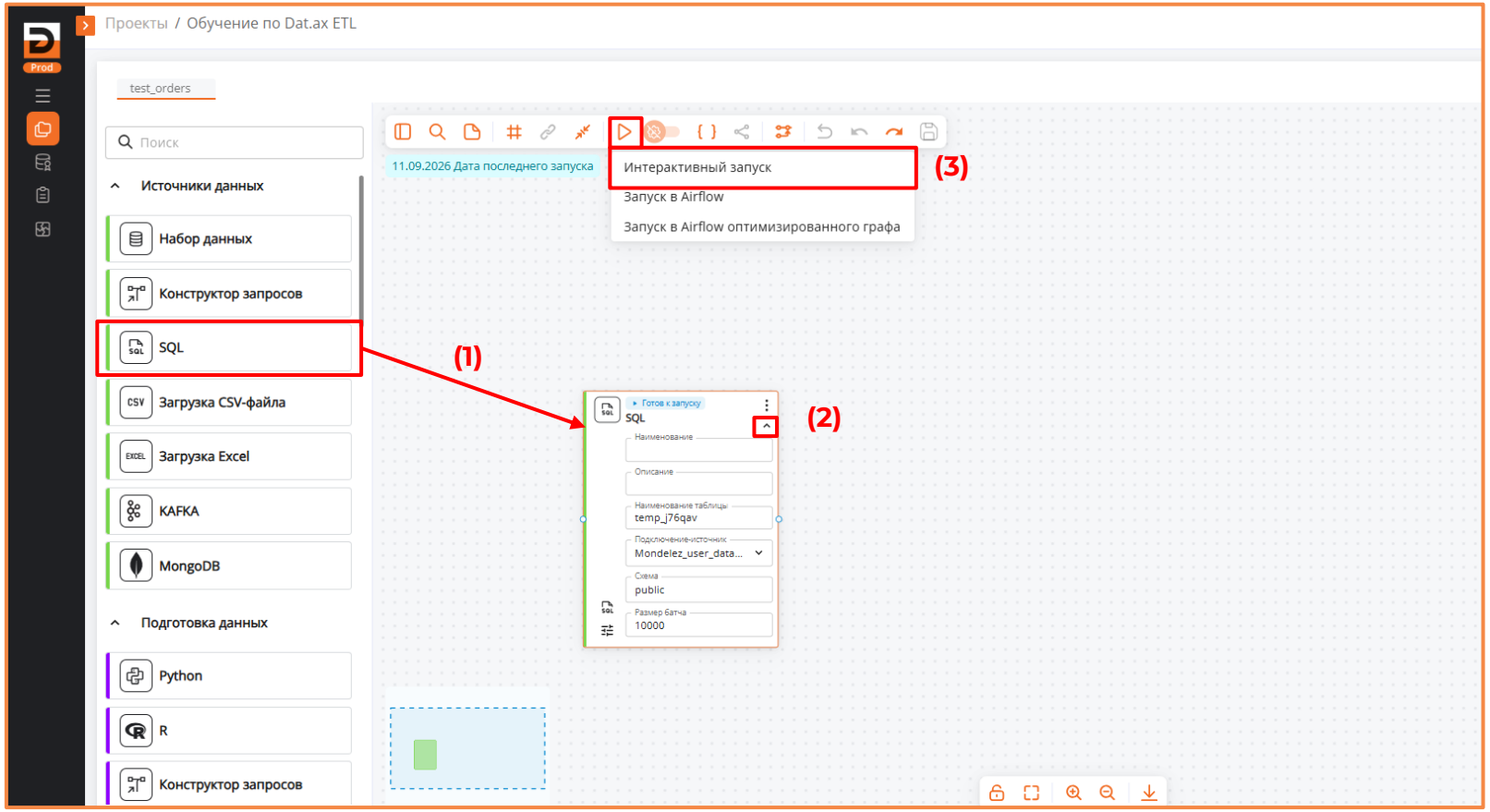
- Нажать на иконку **Настройки пайплайнов** в меню управления пайплайном;
- В открывшемся окне **Настройки пайплайнов** в строке с интересующим пайплайном, нажать иконку Действия строки --- и в выпадающем списке;
- В открывшемся окне произвести операции;
- Нажать кнопку **Сохранить**.




**Редактирование пайплайна**

# Теория: Работа с пайплайном

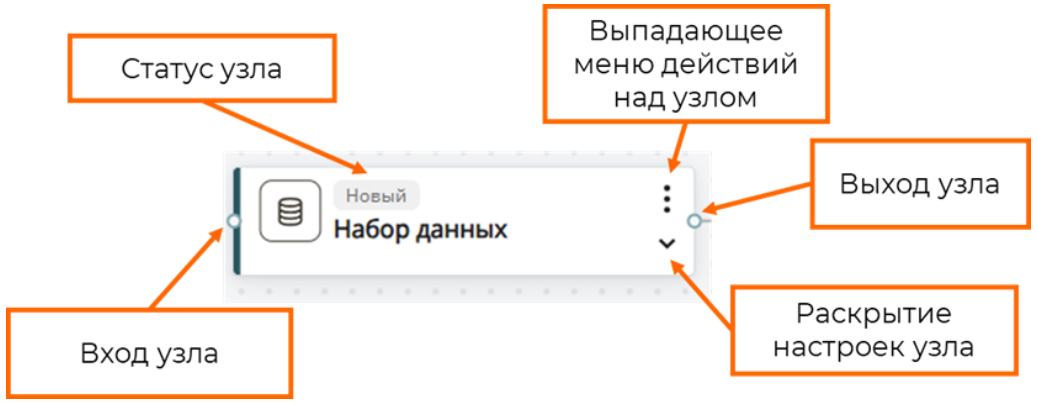
Узлы можно перетаскивать в рабочую область пайплайна и настраивать их параметры (1). После добавления узла его параметры можно раскрыть с помощью стрелки (2). Для запуска всего пайплайна используется кнопка в верхнем меню (3).




**Рабочая область пайплайна**

# Теория: Работа с пайплайном

Узел каждой группы имеет свое цветовое обозначение и настройки. Общие элементы узлов отображены на рисунке ниже.




**Структура узла**

Каждый узел имеет статус. Статус отображается над наименованием узла. Узлы могут иметь следующие статусы:

Новый — При добавлении узла в рабочее поле или сбросе статуса узла.

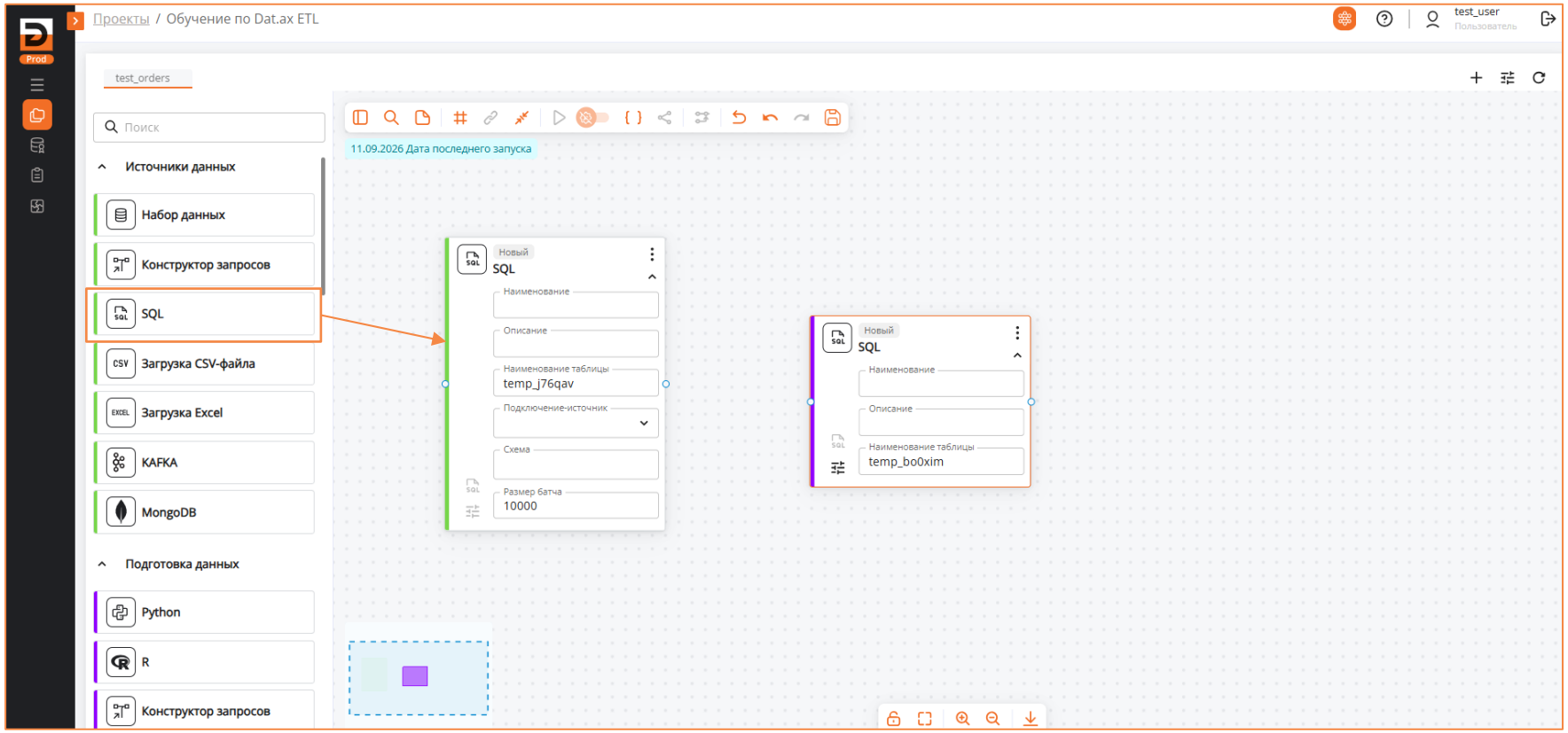
Готов к запуску — После сохранения пайплайна без ошибок валидации;

Выполнен — После успешного выполнения узла;

Ошибка — При неуспешном сохранении пайплайна или выполнении узла. Описание ошибки доступно при раскрытии

# Теория: Работа с пайплайном

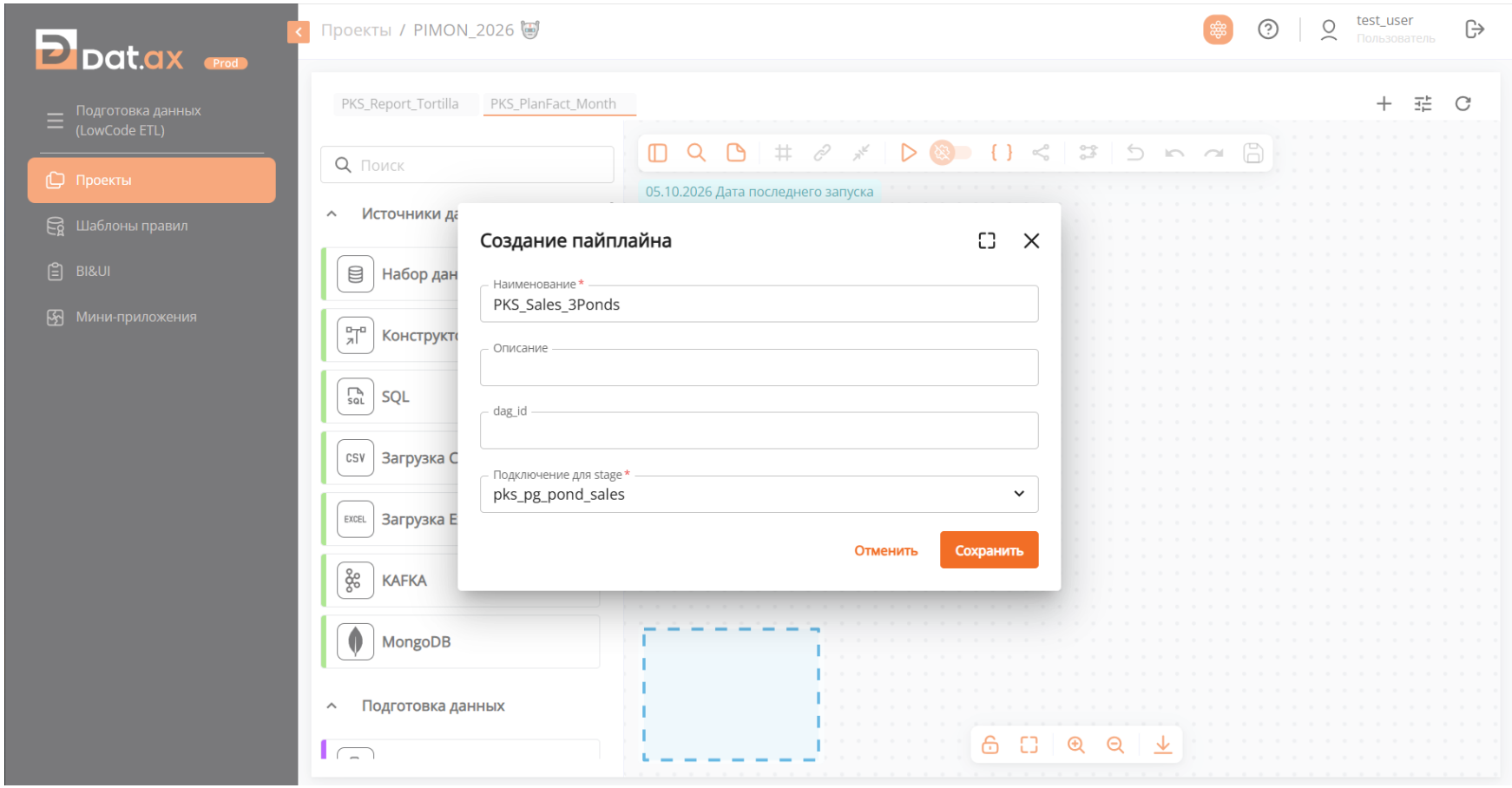
Зелёные узлы категории **Источники данных** переносят данные из указанного подключения в Stage-подключения по частям. **Stage-подключение** используется для промежуточного хранения результатов выполнения узлов. Фиолетовые узлы из группы **Подготовка данных** работают только в рамках Stage-подключения и не выполняют перенос данных по частям, поэтому работают быстрее.




**Рабочая область пайплайна**

# Практика: Создание пайплайнов для отчетности Тортил

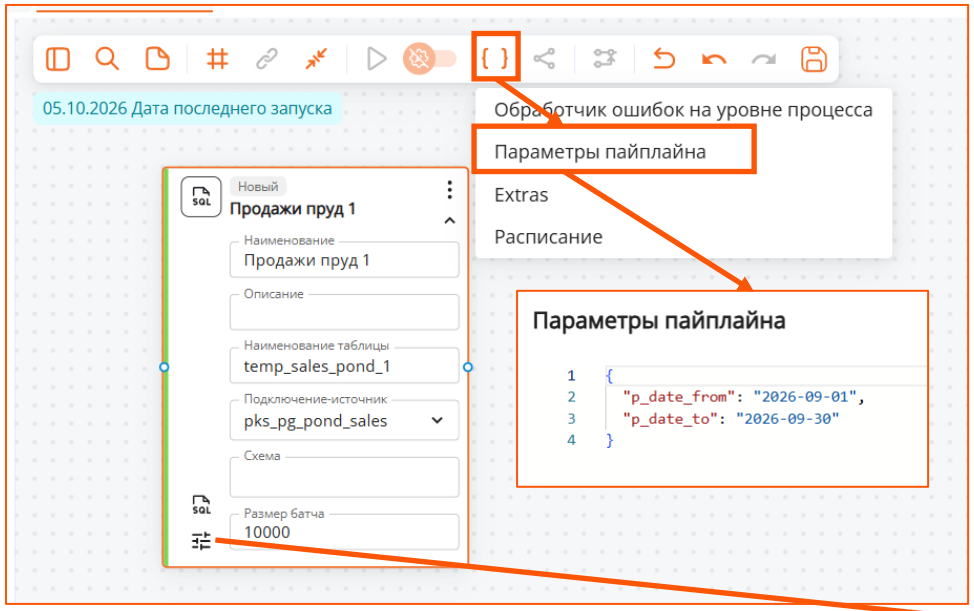
Папа Карло создает самый главный пайплайн проекта «PKS_Report_Tortilla», а также вкладки других пайплайнов под запросы отделов.




**Создание пайплайна**

# Практика: Создание отчета о продажах

Итак, проект создан. Первые пайплайны готовы и ждут наполнения. Папа Карло берется за заявку от Дуремара по подготовке отчета о продажах трех прудов за прошедший период.
Для этого он задает даты начала и окончания периода в параметры пайплайна и собирает 3 первоначальных узла с продажами каждого пруда.



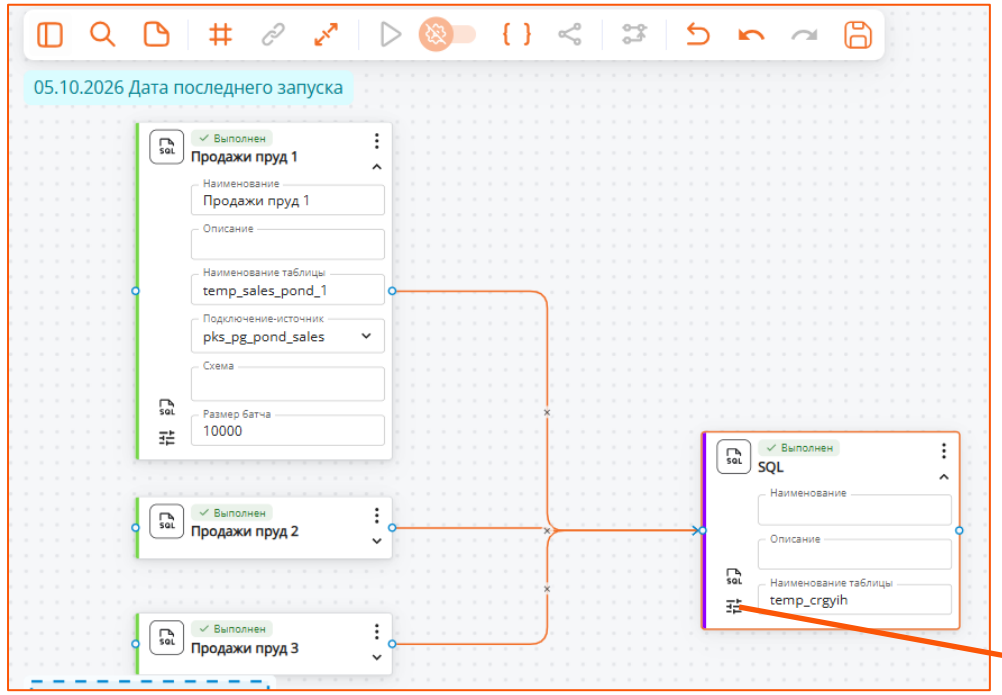


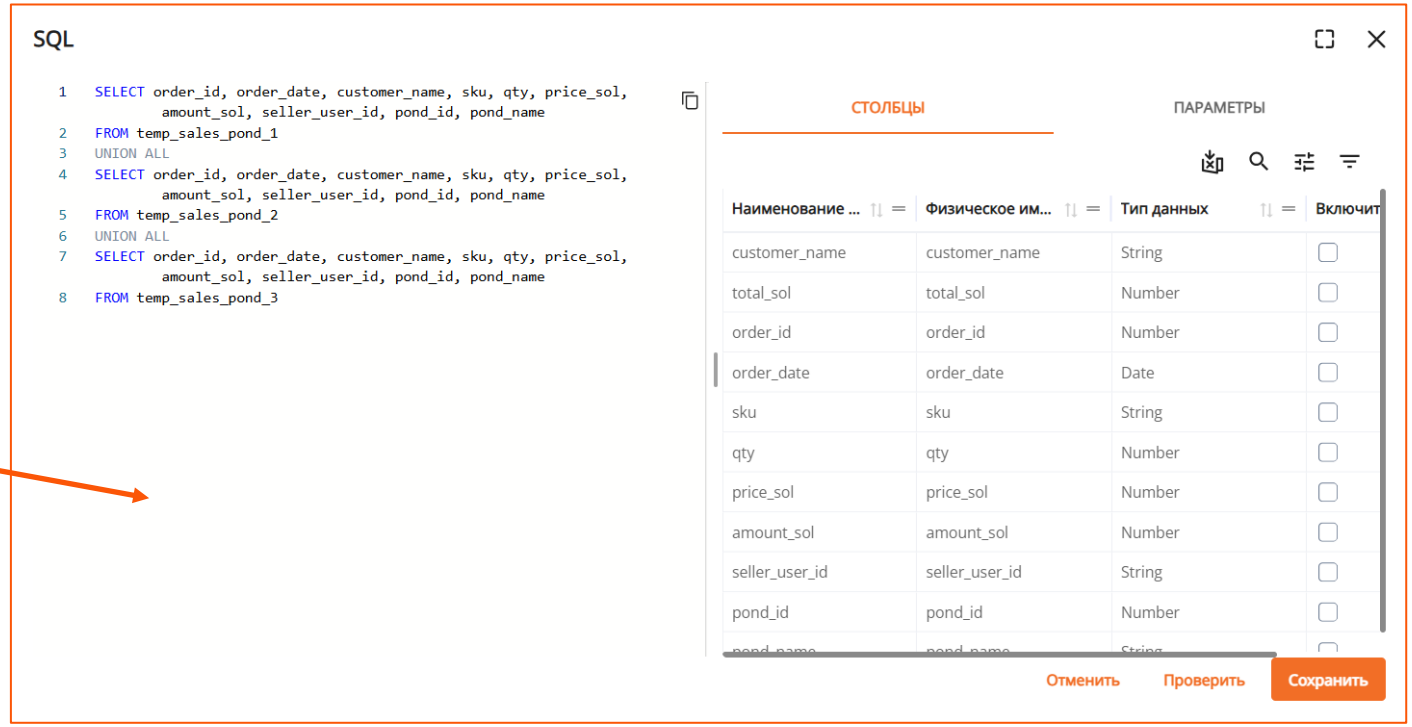


Благодаря параметрам, даты отчетного периода будут передаваться во все три узла.
В дальнейшем, можно будет настроить динамический расчет дат периода от текущей даты, но об этом позже.

# Практика: Создание отчета о продажах

Теперь можно объединить продажи трех прудов за один период с помощью узла обработки данных SQL или специализированного узла UNION.

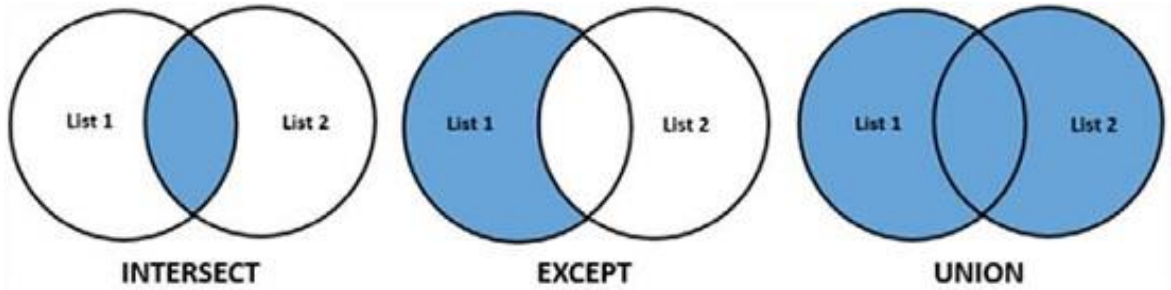


# Теория: Union

Узел **Union** позволяет проводить операции объединения, пересечения и вычитания таблиц, используемые в процессе подготовки данных, а именно:

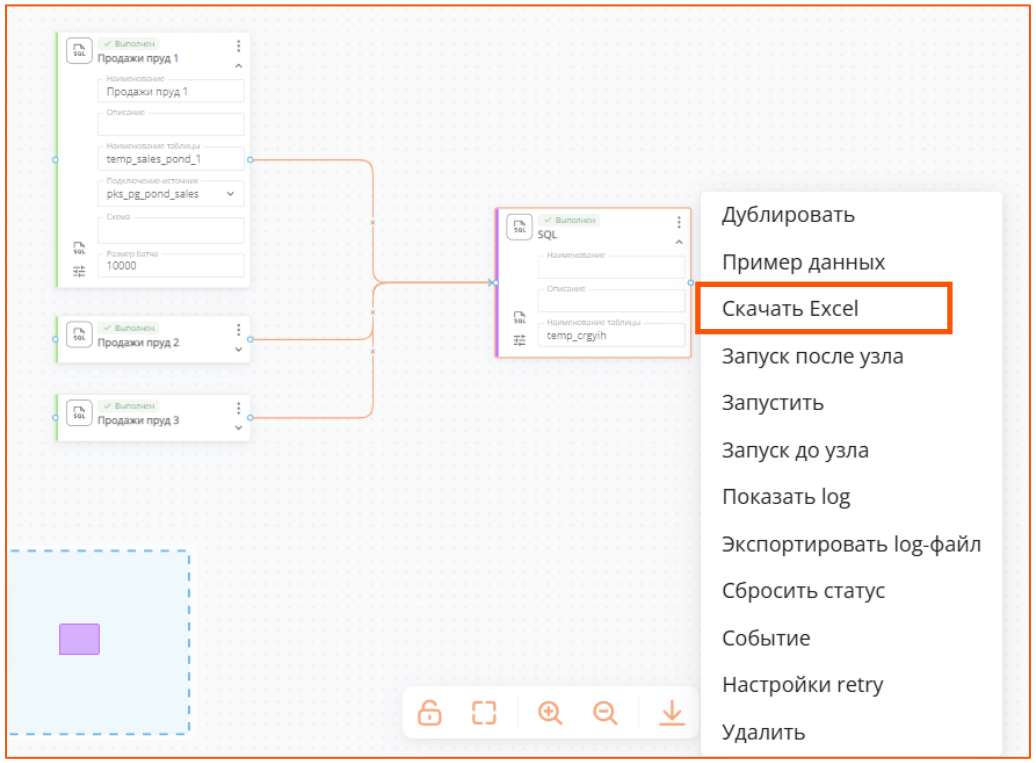
- **UNION** – объединение результатов присоединяемой таблицы к главной без дублирования данных;
- **UNION ALL** – объединение всех результатов присоединяемой таблицы к главной (включая дубликаты);
- **INTERSECT** – возвращение всех строк, одновременно содержащихся в главной и присоединяемой таблицах без дублирования информации;
- **INTERSECT ALL** – возвращение всех строк, одновременно содержащихся в главной и присоединяемой таблицах (включая дубликаты);
- **EXCEPT** – возвращение всех строк, которые есть в главной таблице, но отсутствуют в присоединяемой таблице без дублирования информации;
- **EXCEPT ALL** – возвращение всех строк, которые есть в главной таблице, но отсутствуют в присоединяемой таблице (включая дубликаты).

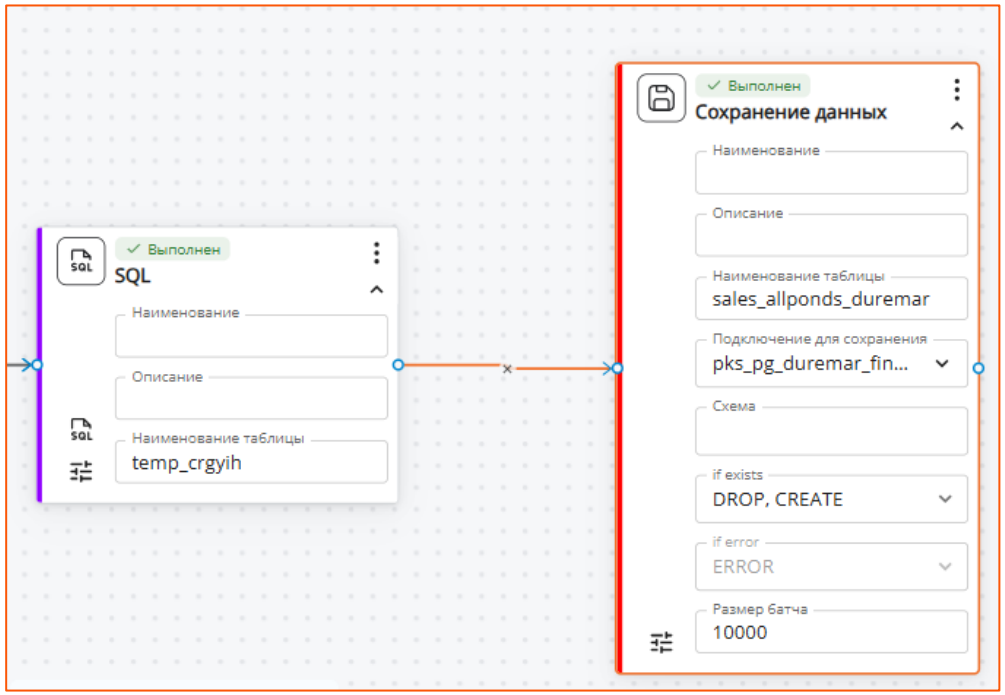


# Практика: Создание отчета о продажах

Завершающий штрих – как Дуремар получит итоговый датасет. Основных варианта два:

1. можно скачать Excel из узла, в котором датасет был сформирован



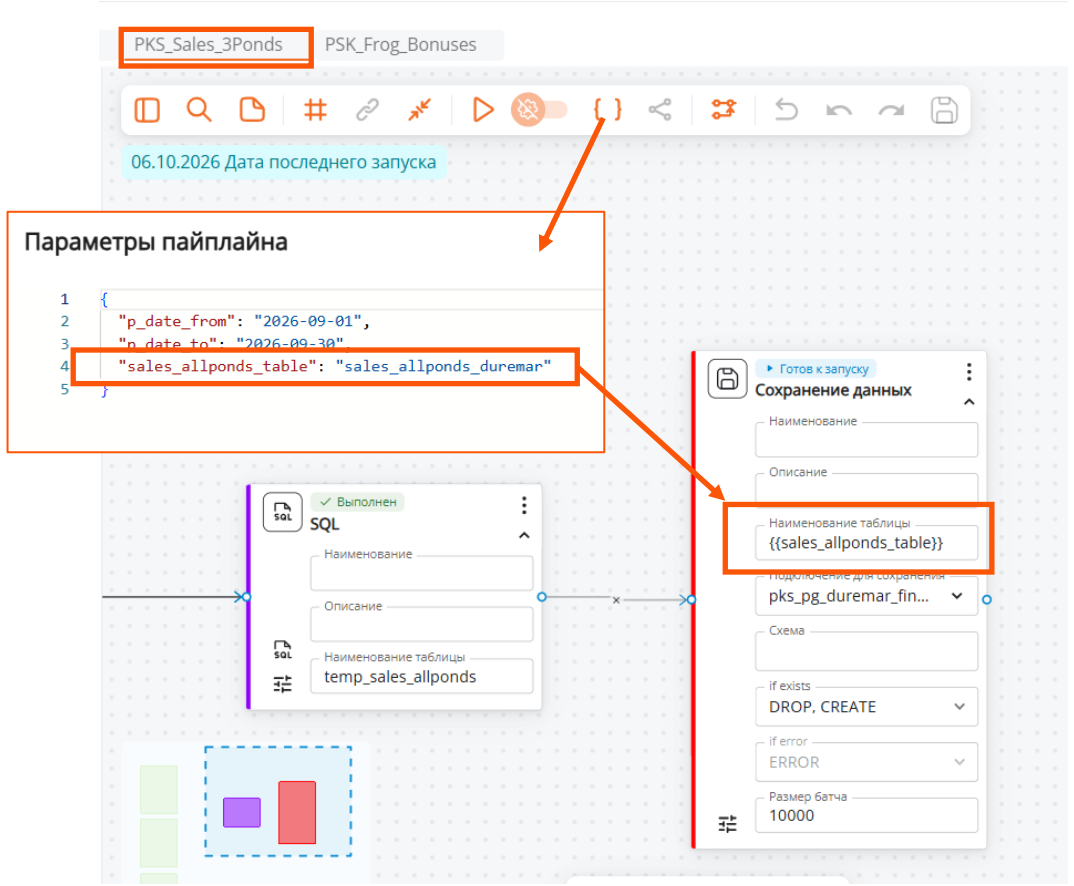

2. либо - записать итоговый датасет в базу, используя узел «Сохранение данных»

# Практика: Бонусы лягушкам

Следующая задача папы Карло – сделать пайплайн для расчета бонусов лягушкам.
У нас уже есть отчет по продажам. В новом пайплайне его можно переиспользовать.

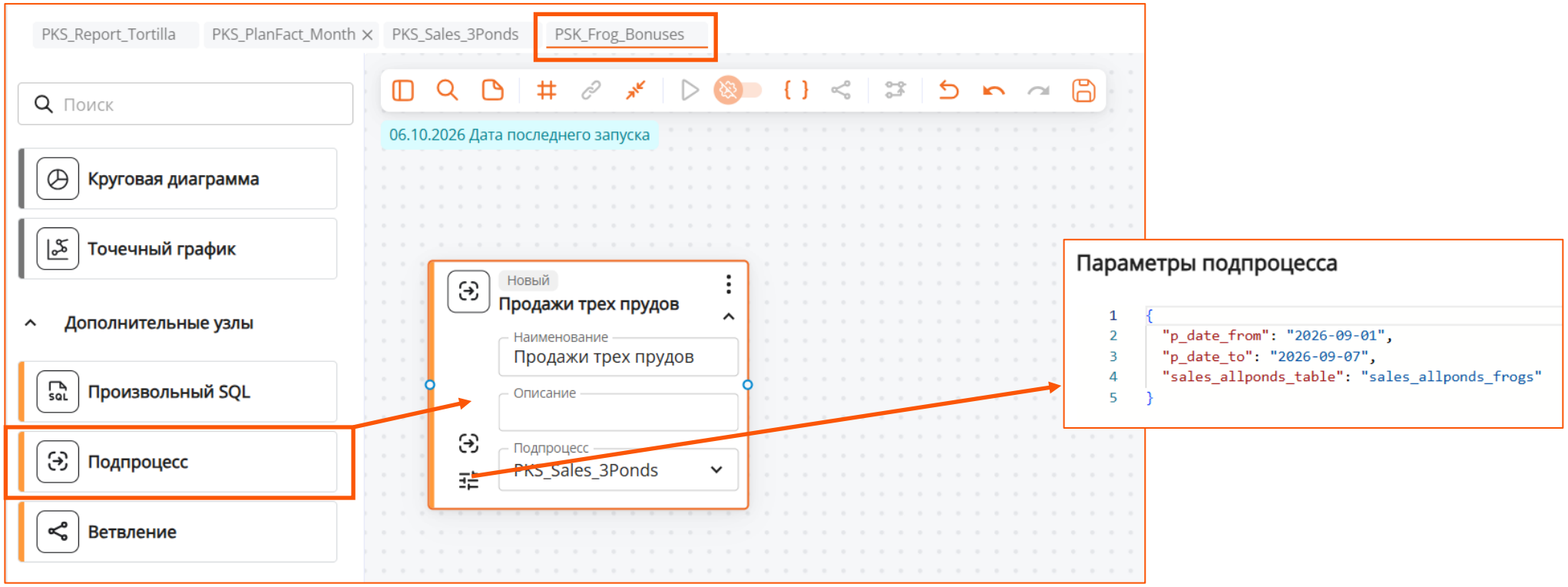
Но лучше немного доработать пайплайн, чтобы блок сохранения получал таблицу для записи из параметров и результаты Дуремара не перезаписывались лягушками.

Мы добавляем в пайплайн параметр имени результирующей таблицы и в узле сохранения данных используем параметр в качестве имени объекта



# Практика: Бонусы лягушкам

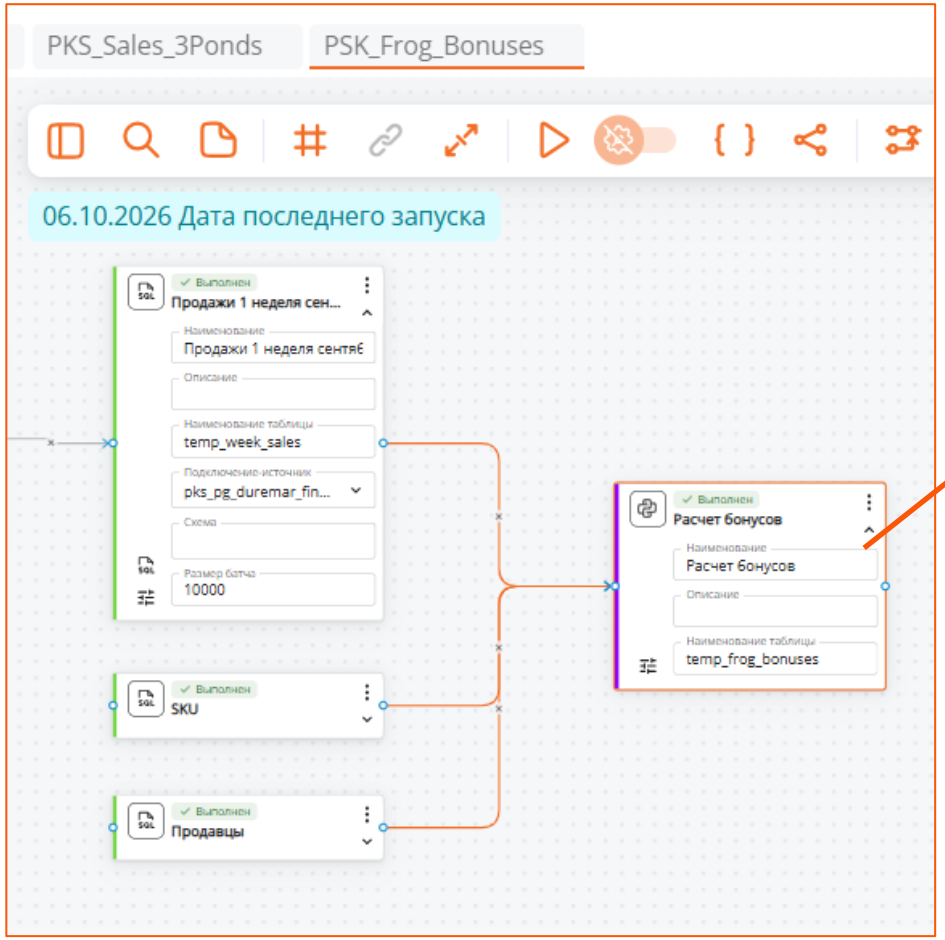
Теперь можно приступать к формированию пайплайна расчета бонусов лягушкам.
Параметры дат отчетного периода и имя таблицы (которая наполняется субпроцессом) можно передавать в узел субпроцесса напрямую, не трогая параметры исходного пайплайна.




Если бы лягушкам были нужны продажи не за весь сентябрь, а за последнюю неделю, они могли бы их получить в таблицу sales_allponds_frogs следующим образом.

# Практика: Бонусы лягушкам

Теперь папа Карло добавляет справочники SKU и Сотрудников для соединения. Нужно произвести вычисление бонусов. Объем датасета позволяет это сделать методами Pandas




**python**

def transform(input_dfs, params, connectors, context):

```python
df_sales = input_dfs['temp_week_sales']
df_rates = input_dfs['temp_sku']
df_frogs = input_dfs['temp_frogs']

df_bonuses = df_sales\
    .merge(df_rates[['sku', 'bonus_pct']], on='sku', how='inner')\
    .assign(bonus_sol=lambda d: (d['amount_sol'] * d['bonus_pct'] / 100).round(2))\
    .groupby('seller_user_id', as_index=False)\
    .agg(revenue_sol=('amount_sol', 'sum'), bonus_sol=('bonus_sol', 'sum'))\
    .merge(df_frogs[['user_id', 'frog_name']], left_on='seller_user_id', right_on='user_id', how='left')\
    .drop(columns='user_id')\
    .sort_values('bonus_sol', ascending=False)

return df_bonuses
```

# Теория: Python

В Dat.ax реализована возможность получения, обработки, записи данных и т.д с использованием **Python** узла, находящегося в группе узлов «Подготовка данных». В стандартной сборке есть возможность проводить импорт следующих библиотек:

- aiohttp
- authlib
- cryptography
- lxml
- mako
- orjson
- pillow
- protobuf
- pyasn1
- pyjwt[crypto]
- python-multipart
- requests
- starlette
- tornado
- urllib
- alembic
- asyncio
- boto3
- celery
- confluent-kafka[avro,schemaregistry]
- fastapi
- flower
- gitpython
- great-expectations
- grpcio
- httpx
- impyla
- ipython
- jinja2
- jpype1
- jq
- jsonschema[format-nongpl]
- kafka-python
- kaleido
- kubernetes
- lightgbm
- networkx[default]
- openai
- openpyxl
- oracledb
- pandas
- pandas-dq
- paramiko
- prophet
- psycopg2-binary
- pydantic
- pydantic-settings
- pydoe3
- pyjwt[crypto]
- pymongo
- pymssql
- pypmml
- python-dateutil
- python-multipart
- pyyaml
- qrcode
- redis
- requests
- rpy2
- scikit-posthocs
- sqlalchemy
- Statsmodels
- truststore
- uvicorn
- xmltodict
- zeep
- pymysql
- rison
- pympler
- iuliia
- yandex-tracker-client
- pandas-access
- pyodbc
- smbprotocol
- mdb-parser
- trino[sqlalchemy]

При создании **Python** узла автоматически проводится импорт библиотеки pandas (import pandas as pd).

# Теория: Python

Из узла **Python** можно напрямую подключаться к различным **источникам данных** с помощью конструкции **with connectors [‘source_type’, ‘source_slug’] as client**:

Полный список типов подключений, доступных в конструкции:

- DB (база данных, которая существует в «Источники данных – Подключения»)
- HTTP_BASIC
- HTTP_BEARER_TOKEN
- HTTP_OAUTH2
- S3
- KAFKA
- FTP
- SFTP
- SMTP
- POP3
- IMAP
- AIRFLOW
- GIT
- GENERIC (для использования MongoDB или просто обобщенного подключения)

Для примера, произведем получение заказов из БД:

python

def transform(input_dfs, params, connectors, context):

```python
from sqlalchemy import text
with connectors['DB', 'mdlz_user_data_pg'] as db:
    res = db.execute(text('SELECT * FROM orders'))
    df = pd.DataFrame(res.fetchall(), columns=res.keys())
return df
```

connectors[‘DB’, ‘mdlz_user_data_pg’] будет возвращать подключение, где DB – тип подключения, а mdlz_user_data_pg – slug подключения.

Переменная res будет иметь тип CursorResult, представляющий результат SQL-запроса. Для получения DataFrame результат надо извлечь и получить имена колонок.

# Теория: Python

Теперь попробуем преобразование данных. Узел **Python** похож на узел SQL, но предназначен для написания Python-кода.

Данные с предыдущих узлов можно найти внутри структуры **input_dfs[ ]**. В этой структуре все таблицы, связь с которыми входит в узел Python, хранятся в виде словаря датафреймов. В этом словаре ключ – название таблицы, а значение – сам датафрейм.

В узле **Python** данные обрабатываются средствами языка — например, можно объединять таблицы и рассчитывать новые колонки. Код должен завершаться оператором **return**, который возвращает итоговый датафрейм.

python

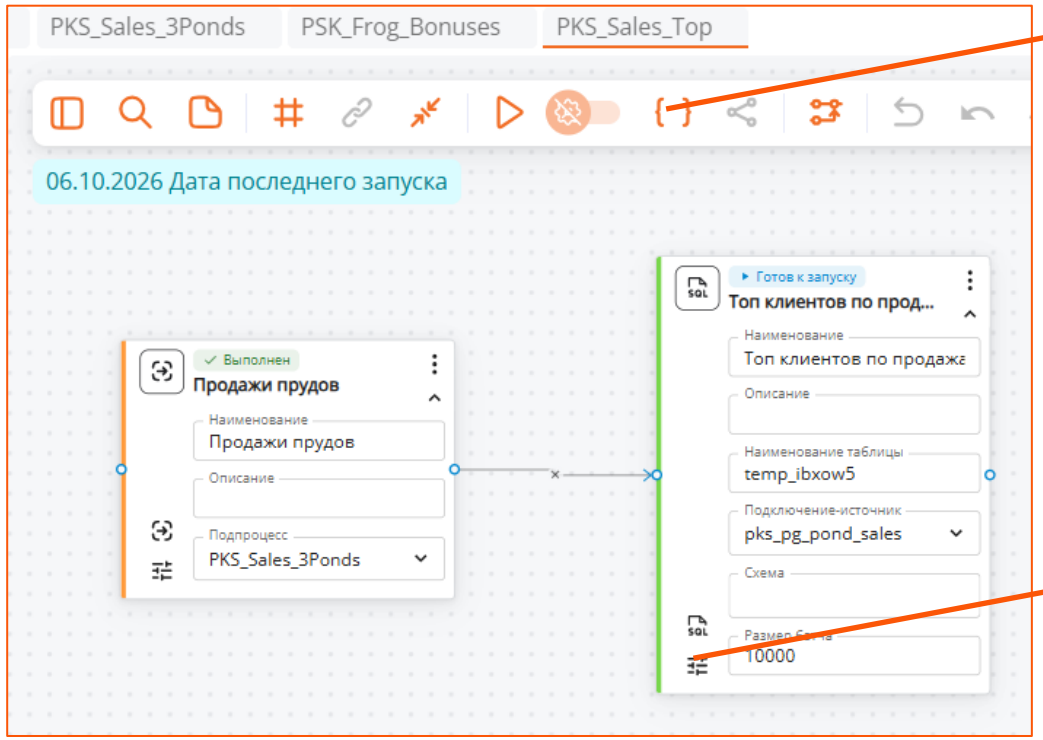
```
def transform(input_dfs, params, connectors, context):
    df_orders = input_dfs['temp_orders_mpv']
    df_customers = input_dfs['temp_customers_vaa']
    df_products = input_dfs['temp_products_mpv']
    df_sales_analitycs = df_orders[df_orders['status'] == 'completed']\
        .merge(df_products, on='product_id', how='inner')\
        .merge(df_customers, on='customer_id', how='inner')\
        .assign(total_sum=lambda d: d['quantity'] * d['price'])\
        [['customer_id', 'city_id', 'product_name', 'total_sum']]
    return df_sales_analitycs
```

| Наименование колонки | Тип | Физическое имя колон... |
|---|---|---|
| customer_id | Number | customer_id |
| city_id | Number | city_id |
| product_name | String | product_name |
| total_sum | Number | total_sum |

Помимо коннекторов и датафреймов, в **Python** узле можно взаимодействовать с объектами: params (словарь с параметрами пайплайна) и context (словарь с Airflow context)

# Практика: Топ клиентов по выручке

Пока папа Карло собирает отчет лягушкам, Арлекин получил задачу от Карабаса-Барабаса выводить топ клиентов по выручке. Поскольку ранее папа Карло уже реализовал пайплайн  с продажами по прудам, задача решается несложно.




**Параметры пайплайна**

```
{
  "sales_allponds_table": "sales_allponds_arlekin"
}
```

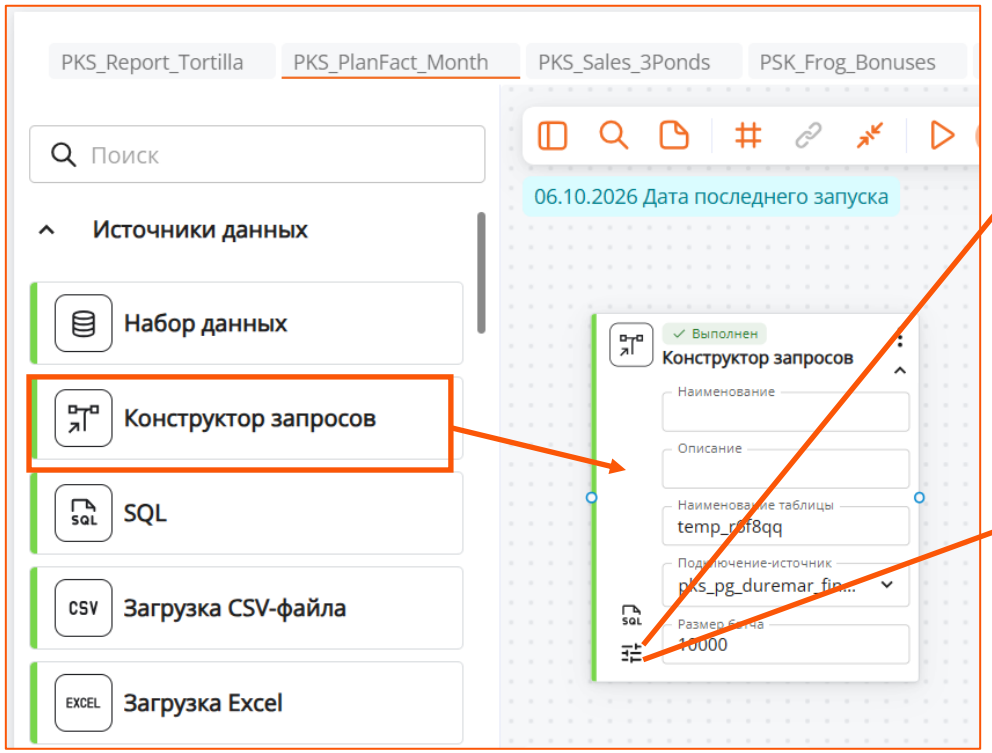
**SQL**

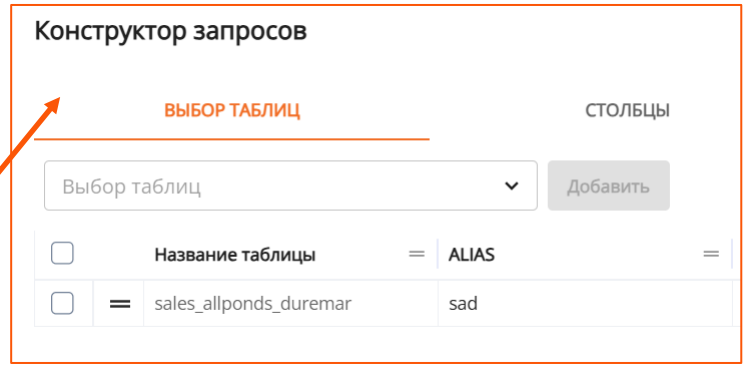
```
SELECT customer_name,
       COUNT(*)         AS orders_cnt,
       SUM(amount_sol) AS total_sol
FROM {{sales_allponds_table}}
GROUP BY customer_name
ORDER BY total_sol DESC
```

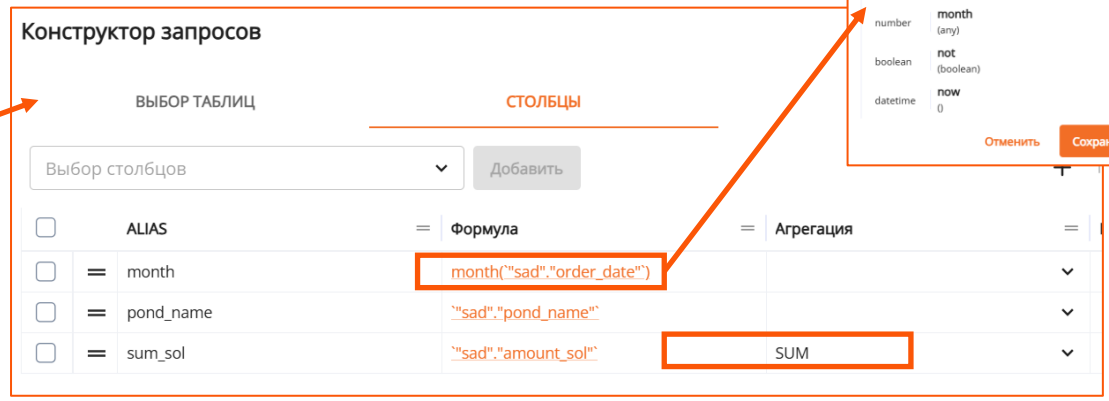
Арлекин решил что хочет чтобы субпроцесс складывал данные продаж в отдельную таблицу, опять же, чтобы не конкурировать с таблицами Дуремара. Он реализует это через параметр пайплайна, который имеет приоритет над параметрами, установленными в субпроцессе по умолчанию.

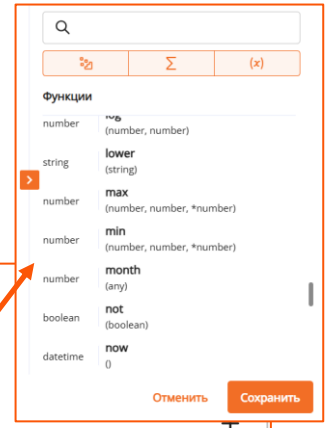
# Практика: План-факт отчет

В это время Дуремару срочно требуется сделать отчет План Факт для Тортиллы. Но ведь он не может обойтись без аналитика и разработчика с их знаниями SQL и программирования. Или может? Он пользуется no-code узлами. Источником становится собранная для него папой Карло таблица с продажами.



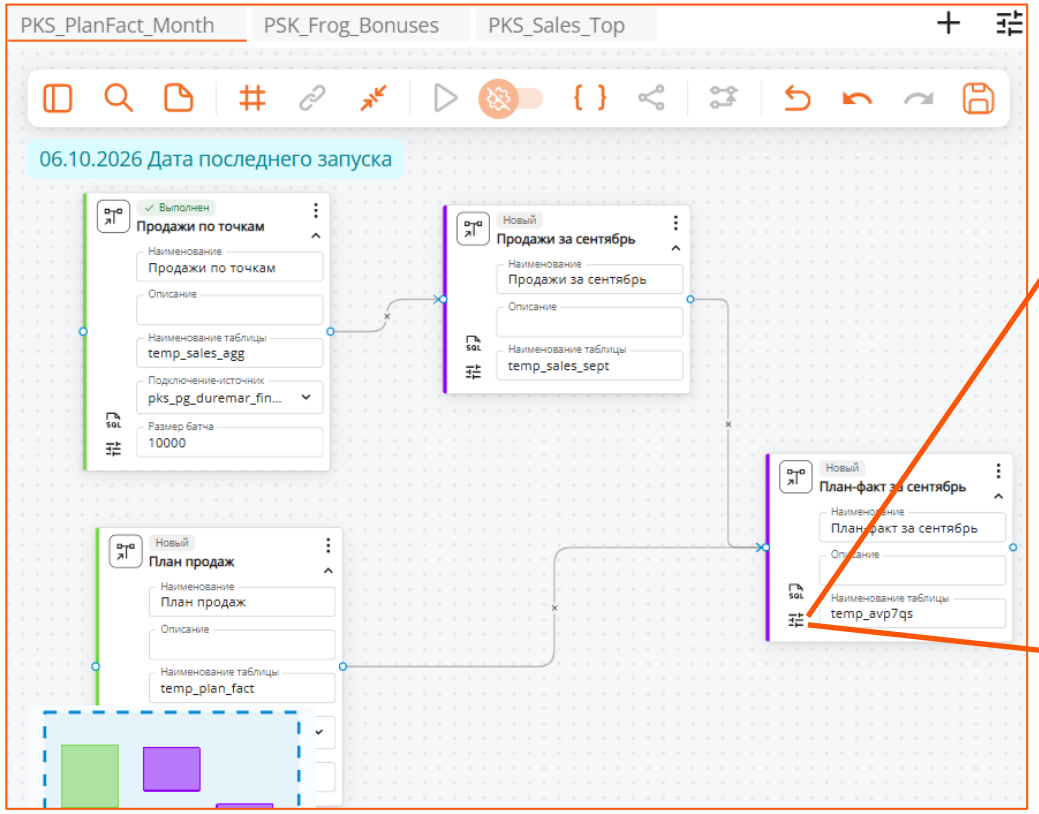



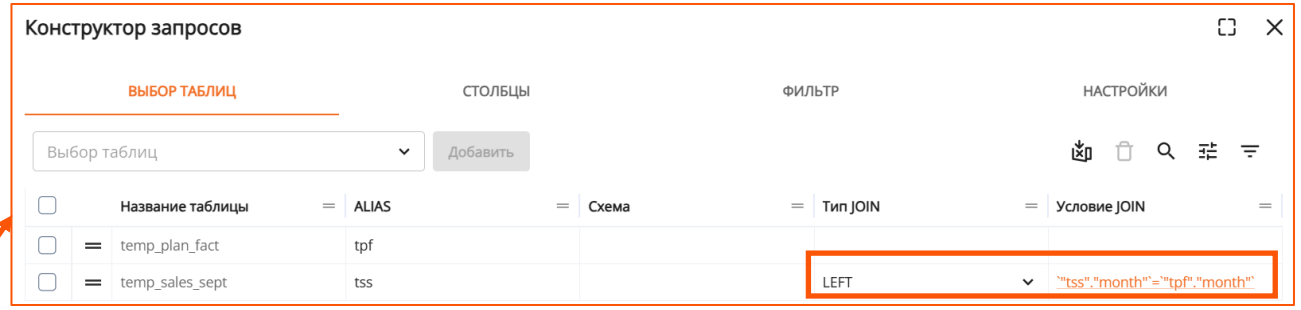



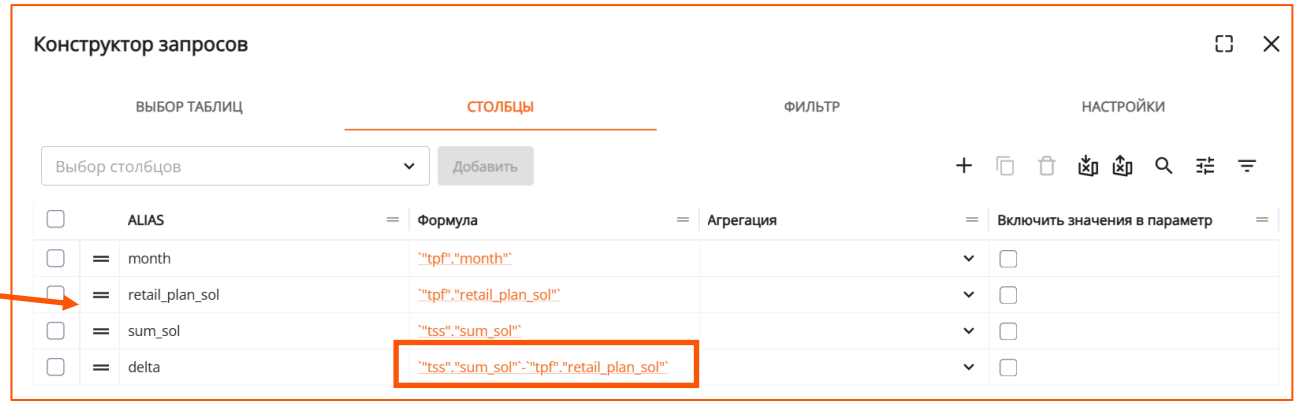

Через Конструктор запросов он выбирает таблицу-источник. На вкладке столбцов преобразует дату, используя богатый инструментарий формул.  Для столбца сумм указывает формулу агрегации.

# Практика: План-факт отчет

Аналогичным образом через Конструктор запросов Дуремар получает загруженный ранее План продаж для розницы из базы. Теперь ему нужно произвести вычисление разницы между планом и фактом. На промежуточном шаге он суммирует продажи точек за месяц. Дальше проводит соединение с таблицей плана и вычисляет дельту.




| month | retail_plan_sol | sum_sol | delta |
|---|---|---|---|
| 9 | 12800 | 11866.99 | -933.01 |

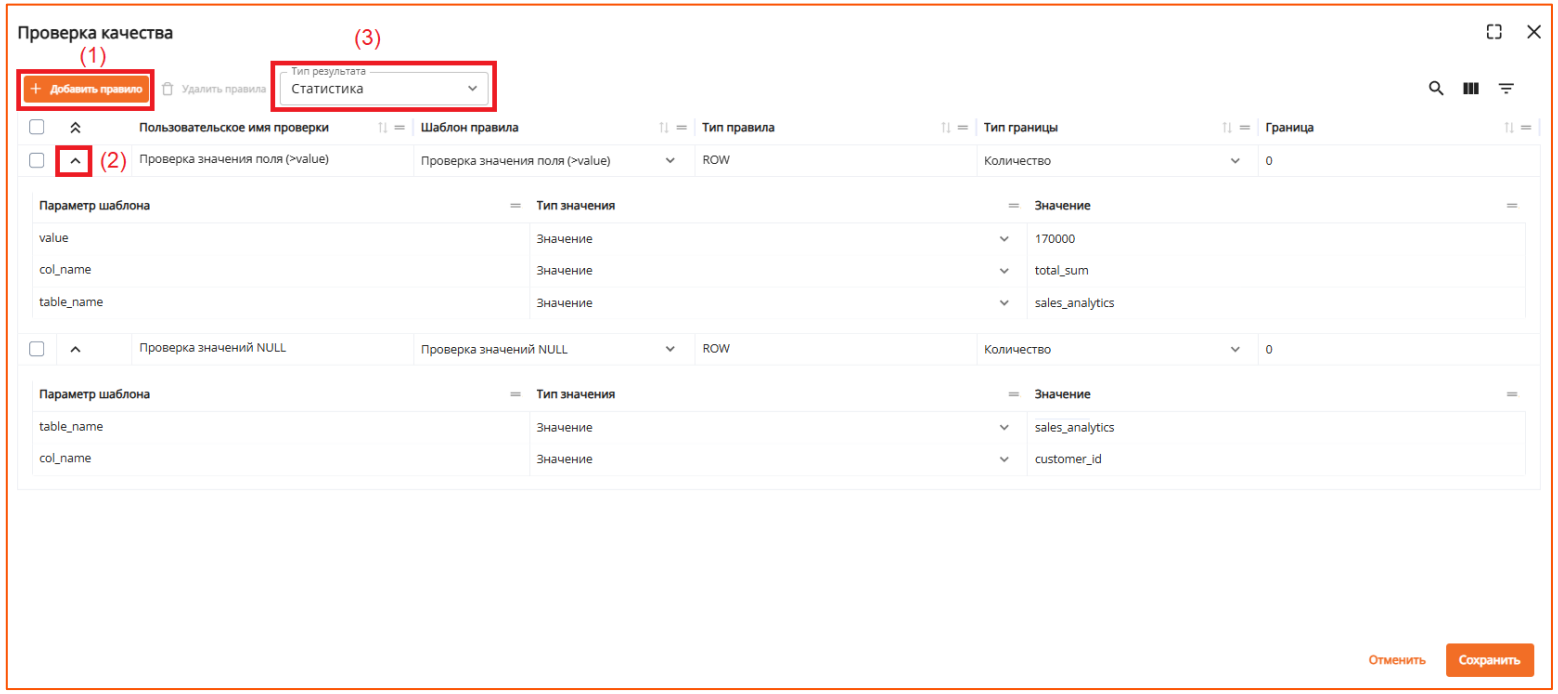
И все это без единой строчки кода.

# Проверка качества данных



# Теория: Проверка качества данных

Узел **Проверка качества** позволяет Пользователю оценить степень соответствия используемых данных стандартным или ожидаемым в процессе подготовки данных. Для проверки качества данных внутри узла используются шаблоны правил.




**Настройка узла Проверка качества**

Для настройки узла необходимо:

1. Нажать кнопку "Добавить правило“
2. Задать пользовательское имя и выбрать шаблон правила из выпадающего списка шаблонов, а также тип границы (количество/процент) и границу, после чего раскрыть шаблон для задания параметров.
3. выбрать тип результата – Ошибки/Статистика. В результате работы узла создаются обе таблицы внутри Dat.ax, но выбор типа результата влияет на то, какая таблица пойдет в следующий узел (при необходимости)

# Теория: Проверка качества данных

Результатом работы узла **Проверка качества** является 2 таблицы: таблица Verification Errors со строками, которые не прошли проверку (с полем описания, какую проверку именно), и таблица DQ Statistics, которая выводит результаты проверок в целом (обнаружены ошибки или нет)

Пример данных

VERIFICATION ERRORS | DQ STATISTICS

| customer_id | city | product_name | total_sum | rule_user_desc | rule_template_code |
|---|---|---|---|---|---|
| 204 | Казань | iPad Pro | 179980 | Проверка значения поля (>value) | ROW |
| 201 | Москва | iPhone 15 | 179980 | Проверка значения поля (>value) | ROW |
| 203 | Москва | iPhone 15 | 269970 | Проверка значения поля (>value) | ROW |

**Таблица Verification Errors**

Пример данных

VERIFICATION ERRORS | DQ STATISTICS

| result | rule | type |
|---|---|---|
| FAILED | Проверка значения поля (>value) | ROW |
| PASSED | Проверка значений NULL | ROW |

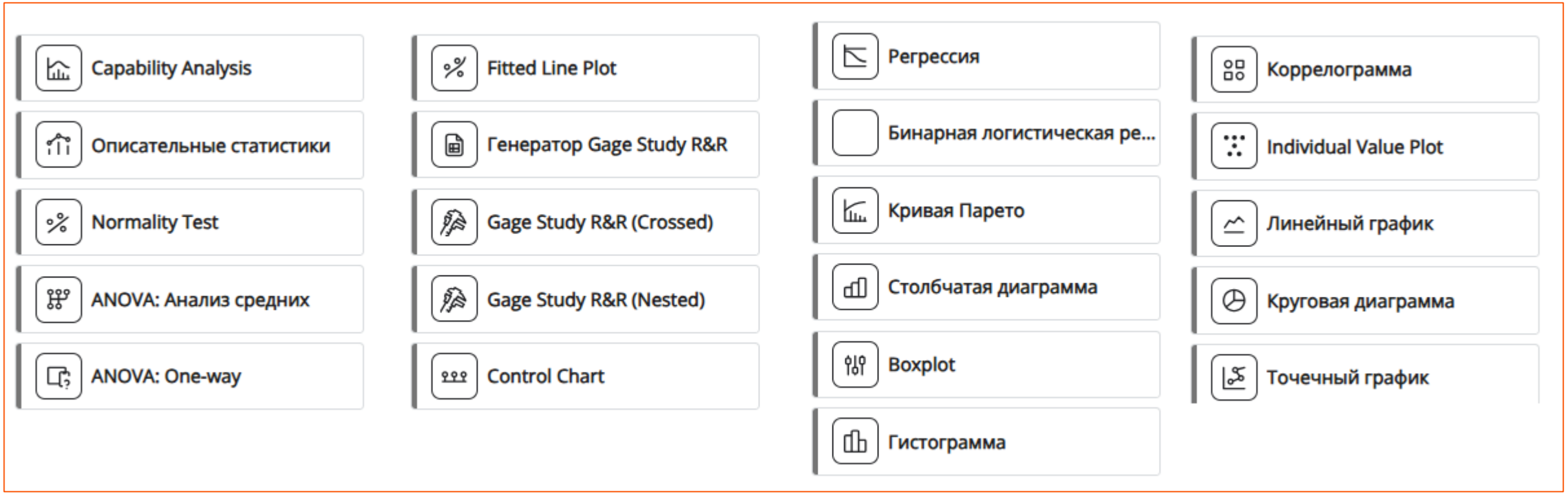
**Таблица DQ Statistics**

# Статистические отчеты



# Обзор узлов Статистические отчёты

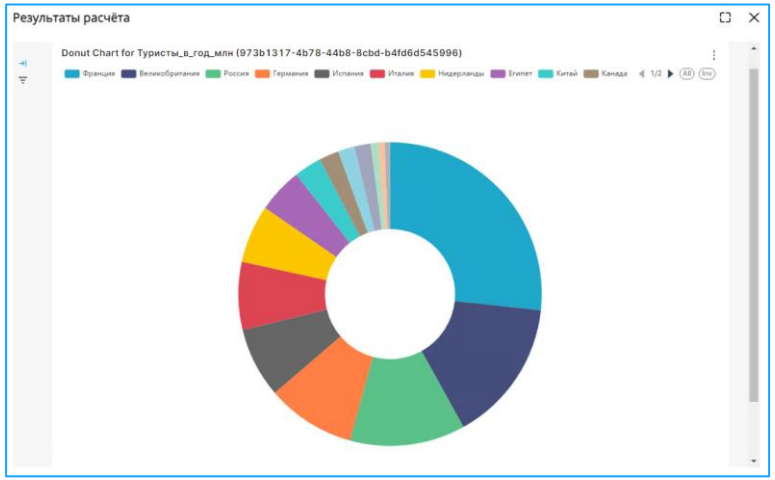
Узлы **Статистические отчёты** предоставляют инструментарий для визуализации и углубленного статистического анализа результатов ETL-процессов. В их арсенале представлены как стандартные средства построения диаграмм и сводок, так и методы сложной аналитики, включая регрессионный и корреляционный анализ.




**Список доступных узлов статистических отчётов**

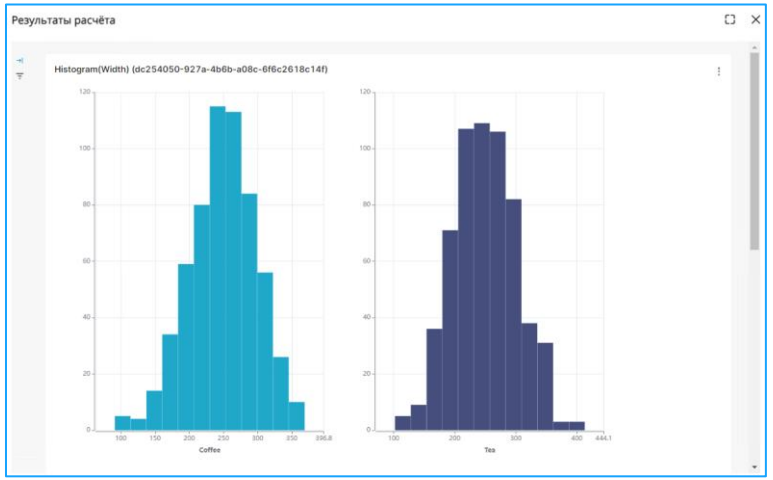
# Обзор узлов Статистические отчёты

С большей частью узлов можно при необходимости ознакомиться в Руководстве пользователя Dat.ax ETL. Рассмотрим базовые узлы статистических отчётов:



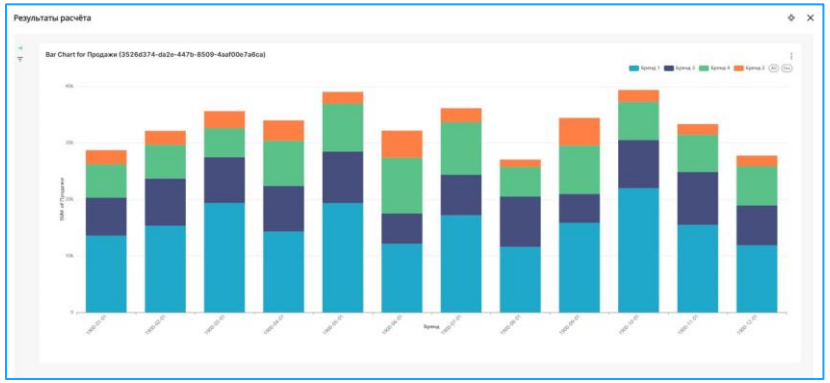

Узел **Круговая диаграмма**

позволяет визуализировать входные данные в виде диаграммы, отображая распределение категориальных данных. Узел Круговая диаграмма используется для сравнения долей данных в каждой категории или группе.




Узел **Гистограмма**

предназначен для анализа формы и распределения данных. В результате выполнения узла отображается гистограмма, которая делит значения выборки на множество интервалов и представляет частоту значений данных в каждом интервале с помощью столбца.

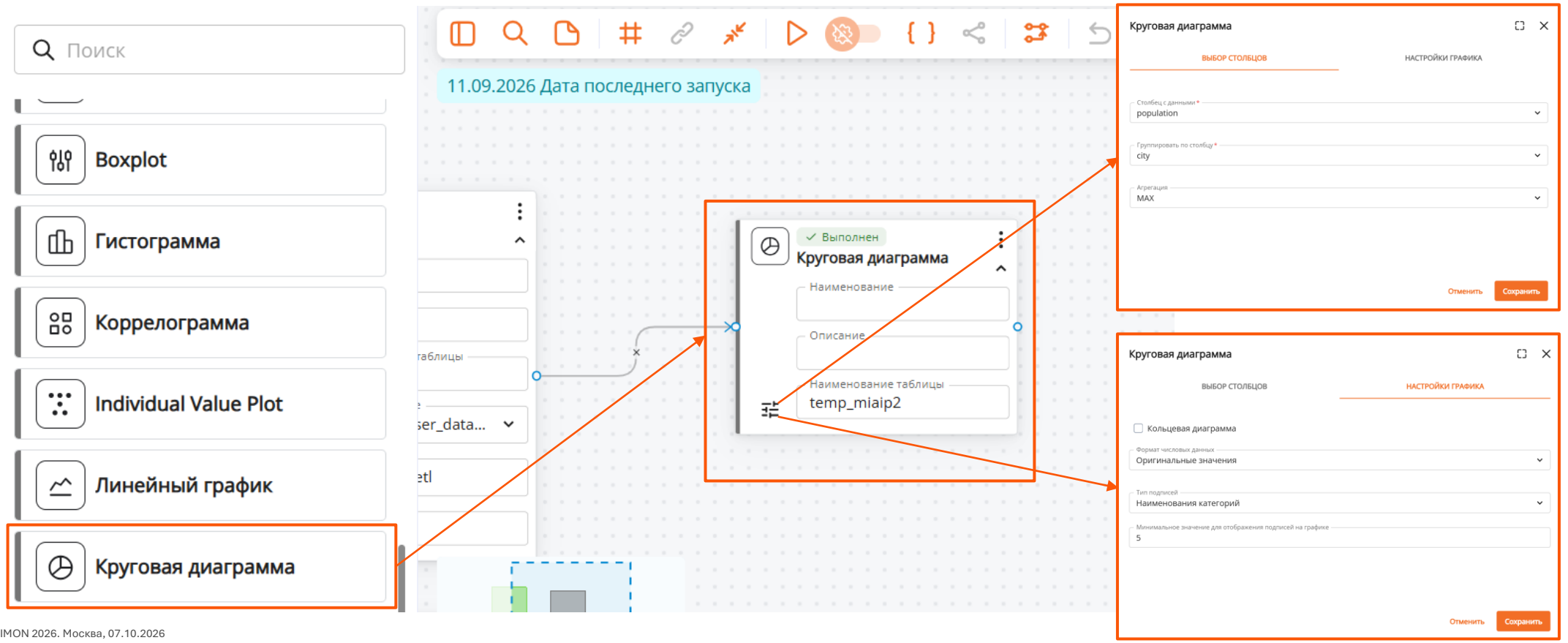



Узел **Столбчатая диаграмма**

позволяет визуализировать входные данные в виде столбчатой диаграммы, демонстрируя распределение категориальных данных.

# Создание узлов Статистические отчёты

Узлы **Статистические отчёты** создаются по аналогии с ранее рассмотренными примерами. При создании узлов, пользователю доступны настройки выбора столбцов входящего набора данных для визуализации, а также настроек отображения.

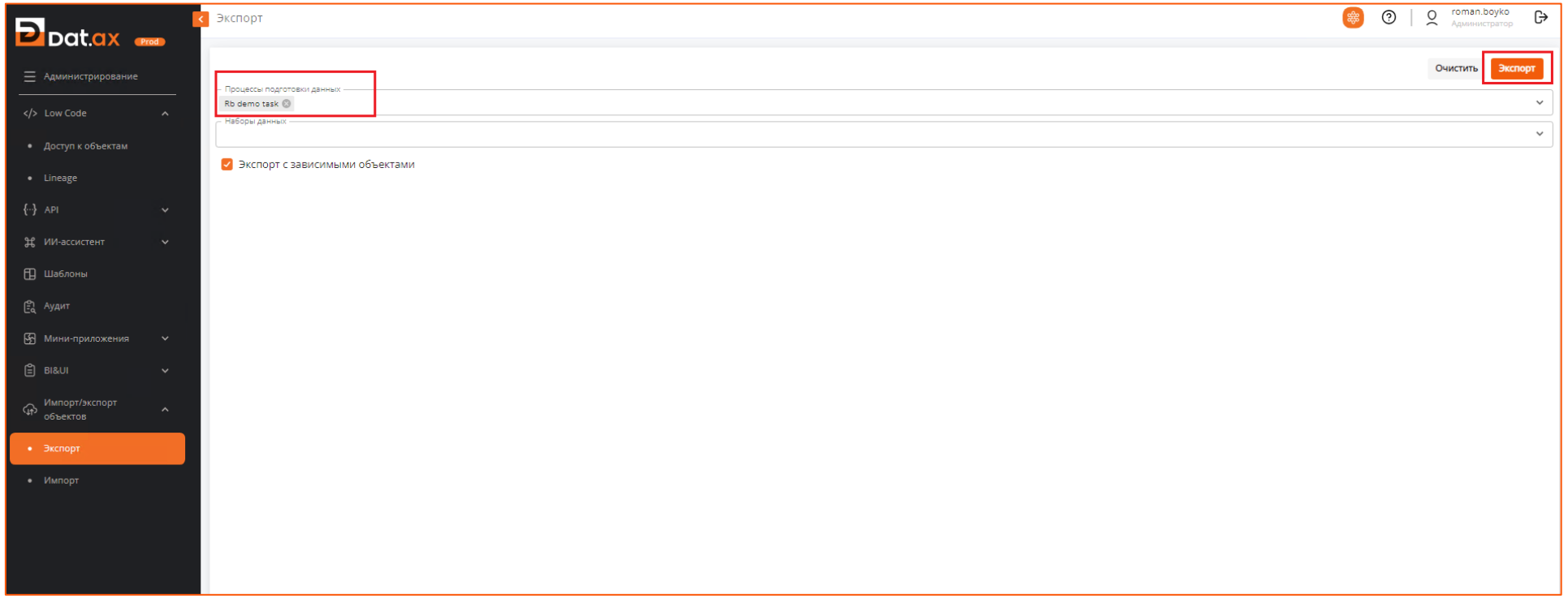


# Экспорт/импорт пайплайнов



# 10.1. Экспорт

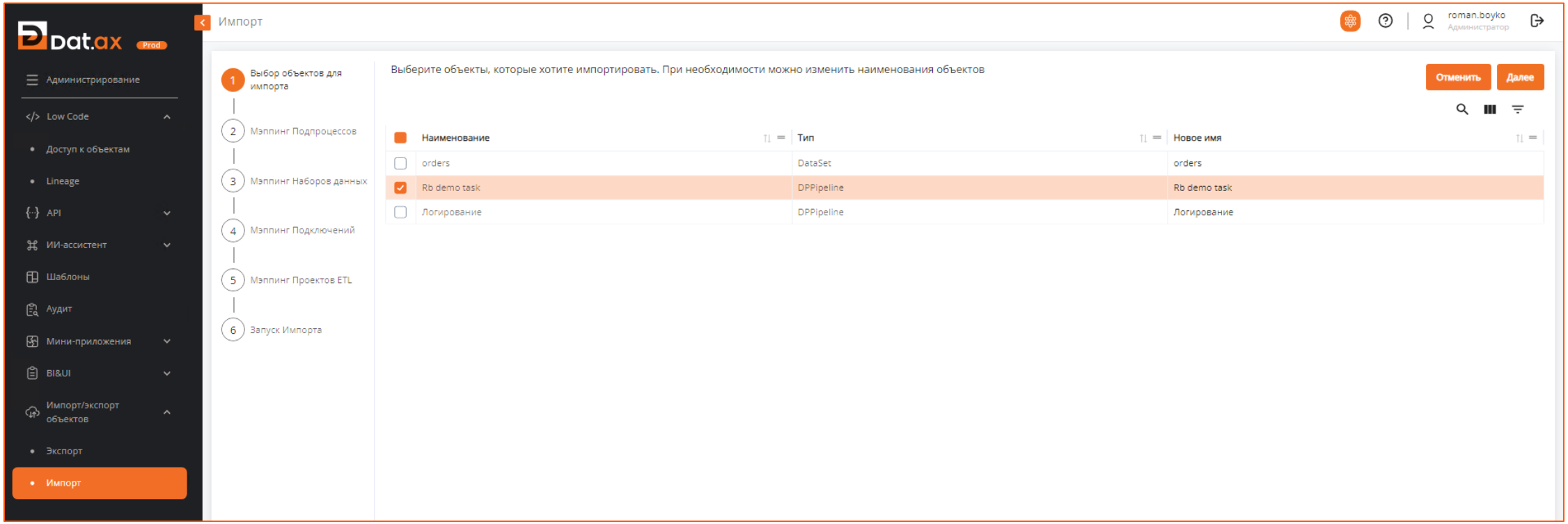
Функции импорт/экспорт объектов предназначены для переноса пайплайна из одной среды в другую в формате JSON. Переход в раздел «Импорт/экспорт»: Администрирование > Импорт/экспорт объектов. Для экспорта объекта необходимо в открывшемся окне в выпадающем списке выбрать процесс подготовки данных (пайплайн). Пользователю доступно выбрать несколько пайплайнов.




**Экспорт пайплайна**

# 10.2. Импорт

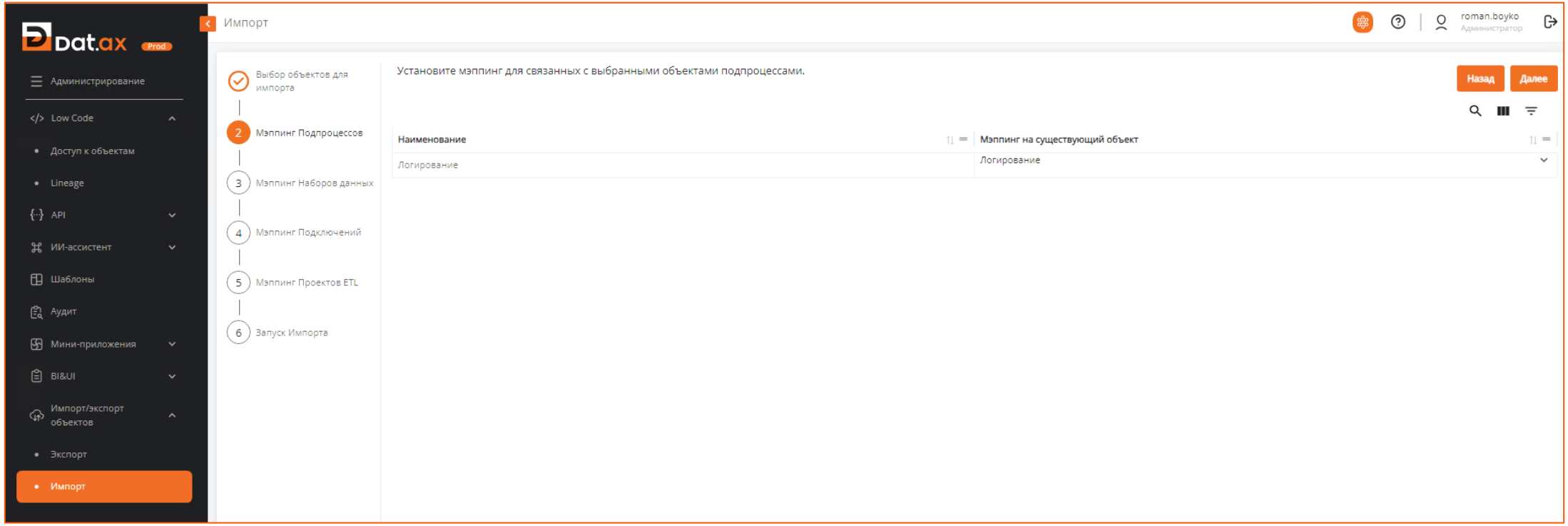
Функция Импорт объектов позволяет Пользователю импортировать объект в Систему в формате JSON для дальнейшей работы. Для импорта объекта необходимо открыть Импорт/экспорт объектов, выбрать Импорт, затем загрузить json файл с пайплайном, после чего выбрать необходимые объекты для импорта.




**Импорт пайплайна**

# 10.2. Импорт

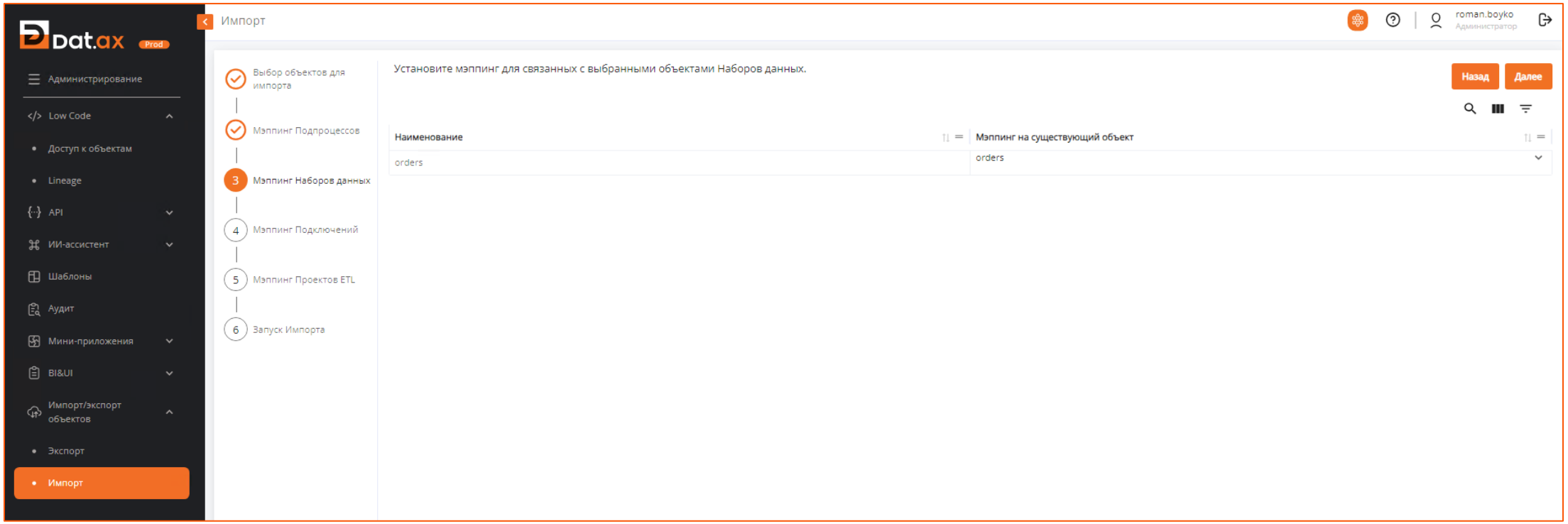
Далее необходимо произвести мэппинг подпроцессов




**Мэппинг подпроцессов**

# 10.2. Импорт

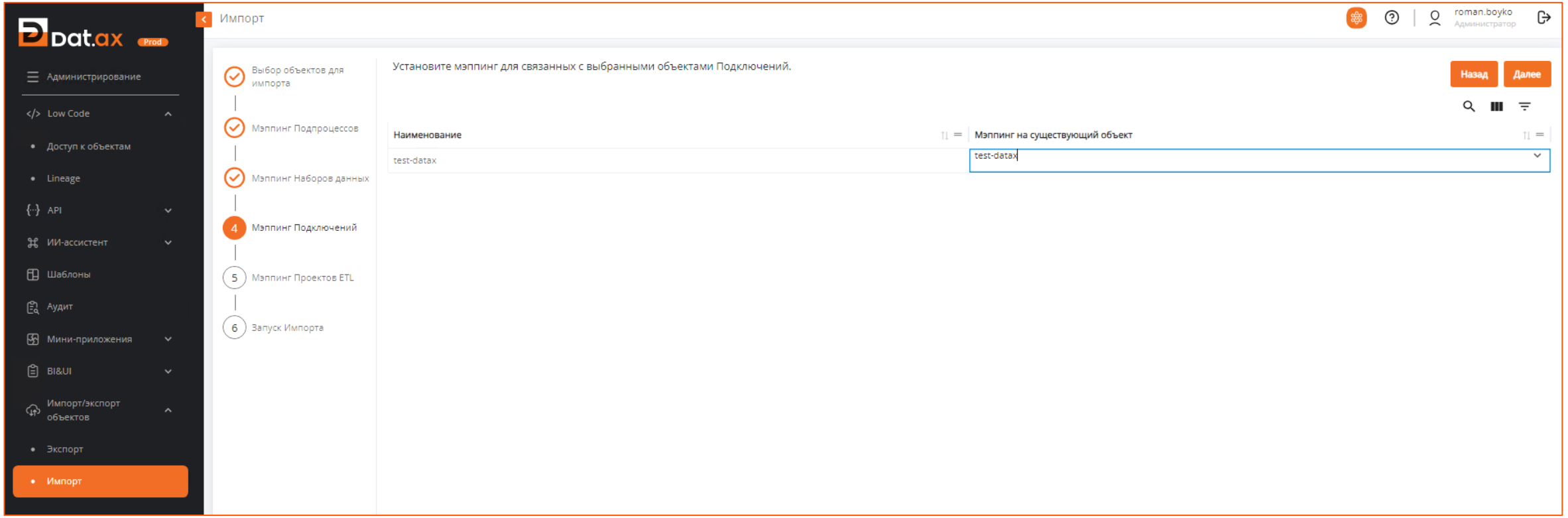
После чего необходимо произвести мэппинг используемых наборов данных




**Мэппинг наборов данных**

# 10.2. Импорт

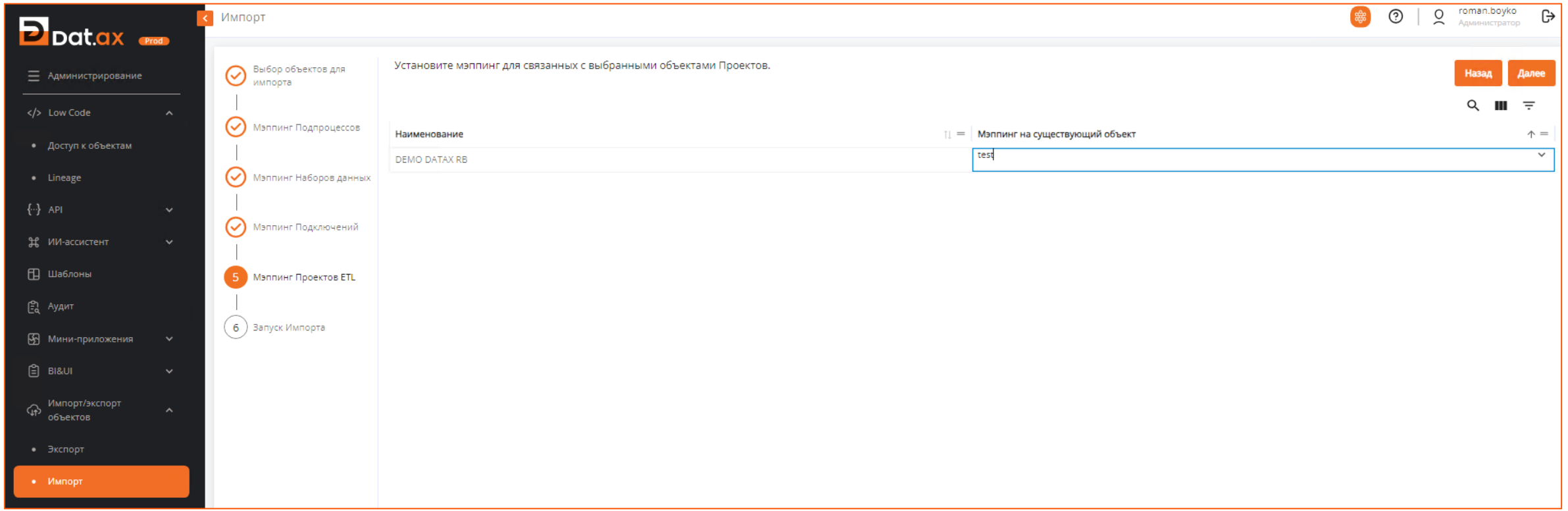
## Далее – мэппинг подключений




**Мэппинг подключений**

# 10.2. Импорт

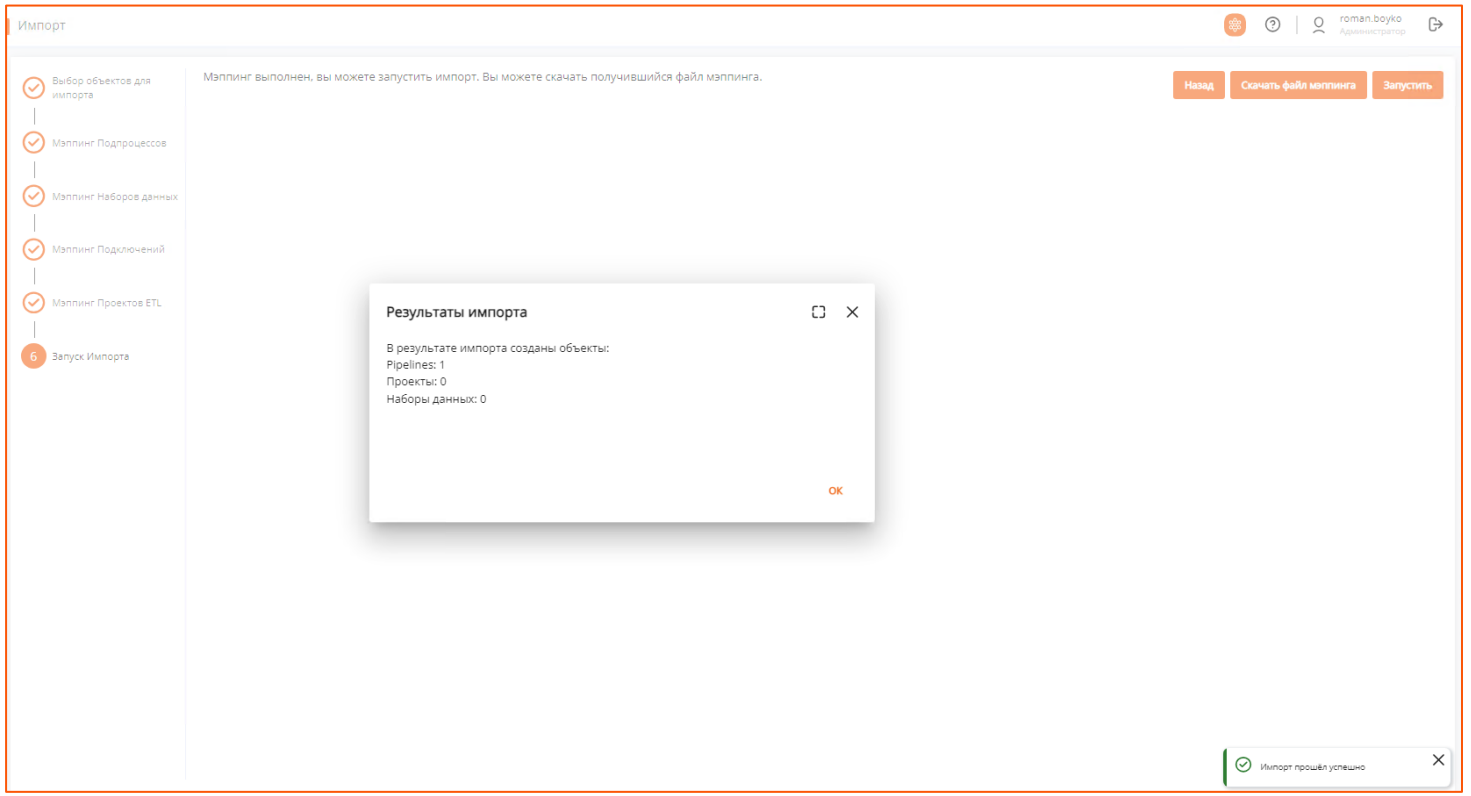
После чего необходимо произвести мэппинг проекта, в котором будет лежать импортируемый пайплайн. Также есть возможность создать новый проект для импортируемого пайплайна.




**Мэппинг проектов**

# 10.2. Импорт

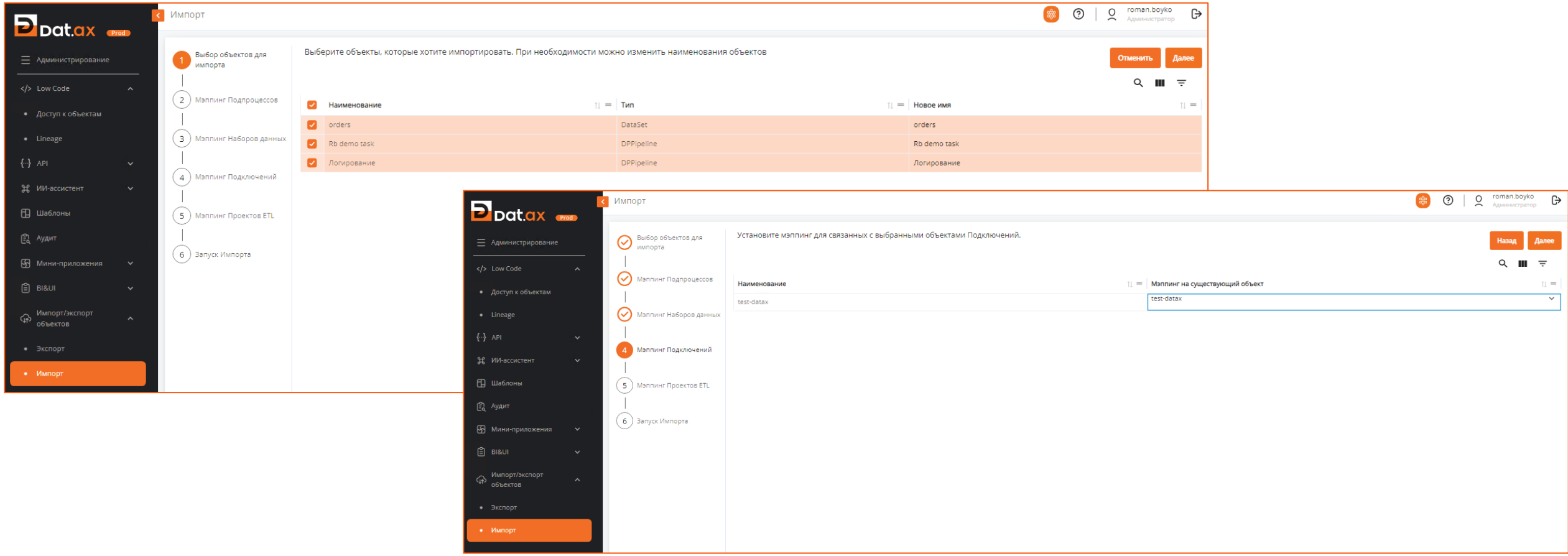
Далее необходимо запустить импорт, после чего в правом нижнем углу появится уведомление об успехе/неудаче, а также откроется окно с результатами импорта




**Результаты импорта**

# 10.2. Импорт пайплайна с зависимыми объектами

В случае, если импортировать сразу все зависимые объекты, шаги 2 и 3 (мэппинг подпроцессов и наборов данных) автоматически пропустятся.



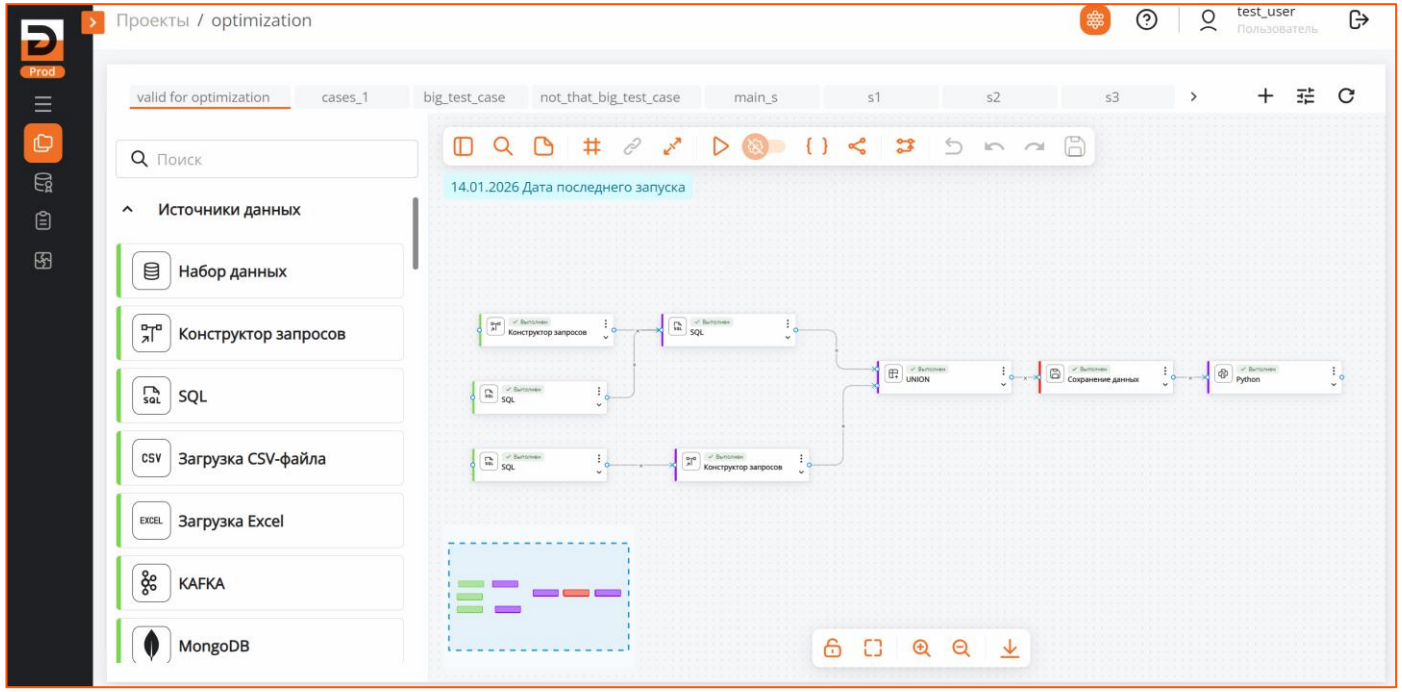

**Пропуск шагов 2,3 при импорте с зависимыми объектами**

# Оптимизация пайплайнов



# Оптимизация пайплайнов

**Оптимизация пайплайнов** – встроенная функция платформы Dat.ax, применяется к SQL-подобным узлам пайплайна при соблюдении критериев оптимизации. В результате запуска создается оптимизированный DAG в Airflow.




При просмотре оптимизированной версии пайплайна система автоматически объединяет несколько SQL-узлов (реализованных через SQL, Набор данных, Конструктор запросов и другие компоненты) в единый SQL-скрипт. Это касается узлов двух категорий:

- Источники данных: SQL, Набор данных, Конструктор запросов;
- Подготовка данных: SQL, Конструктор запросов, UNION, Проверка качества.

# Запуск в Airflow

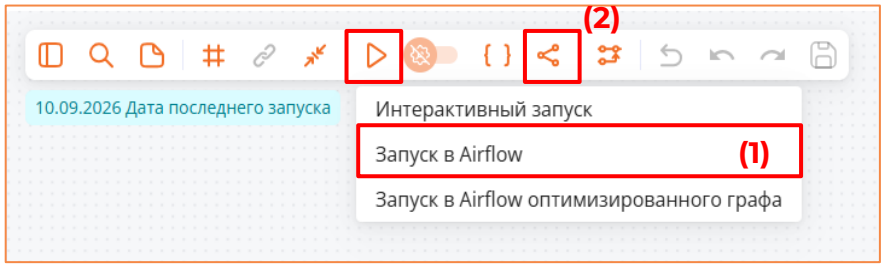


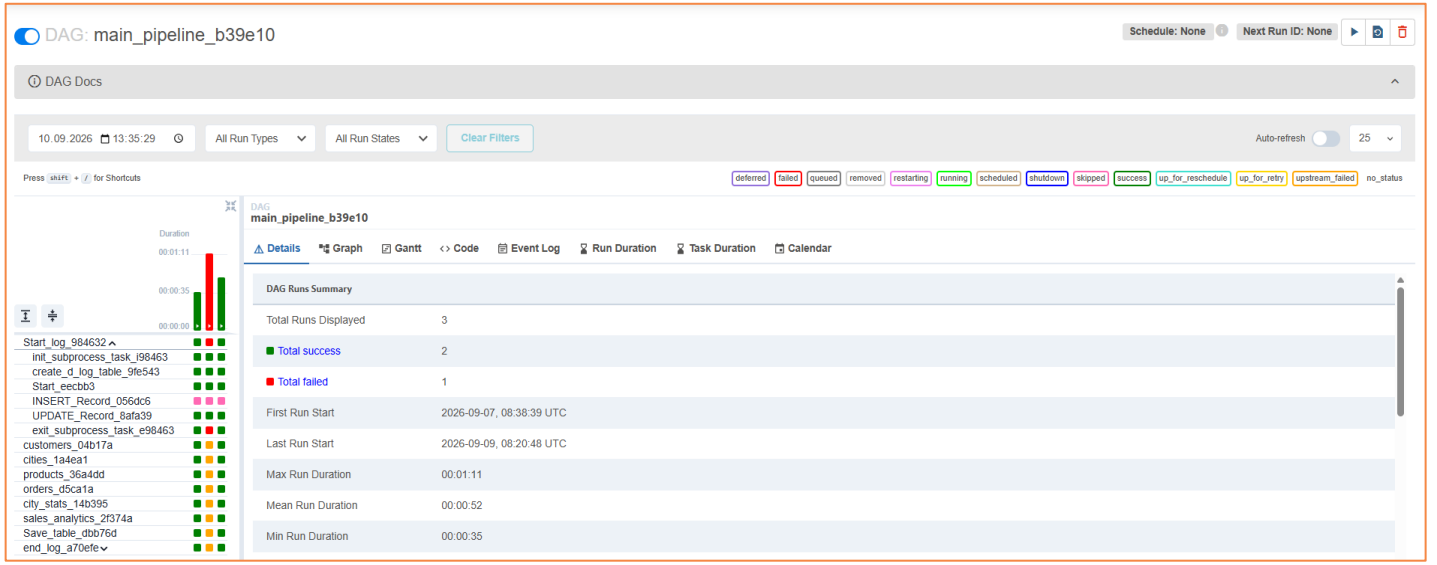
# Запуск в Airflow

Помимо интерактивного запуска пайплайнов,  в Dat.ax предусмотрен второй режим работы – **Запуск в Airflow (1)**, который

используется для регламентных ETL-процессов с выполнением по заданному расписанию.

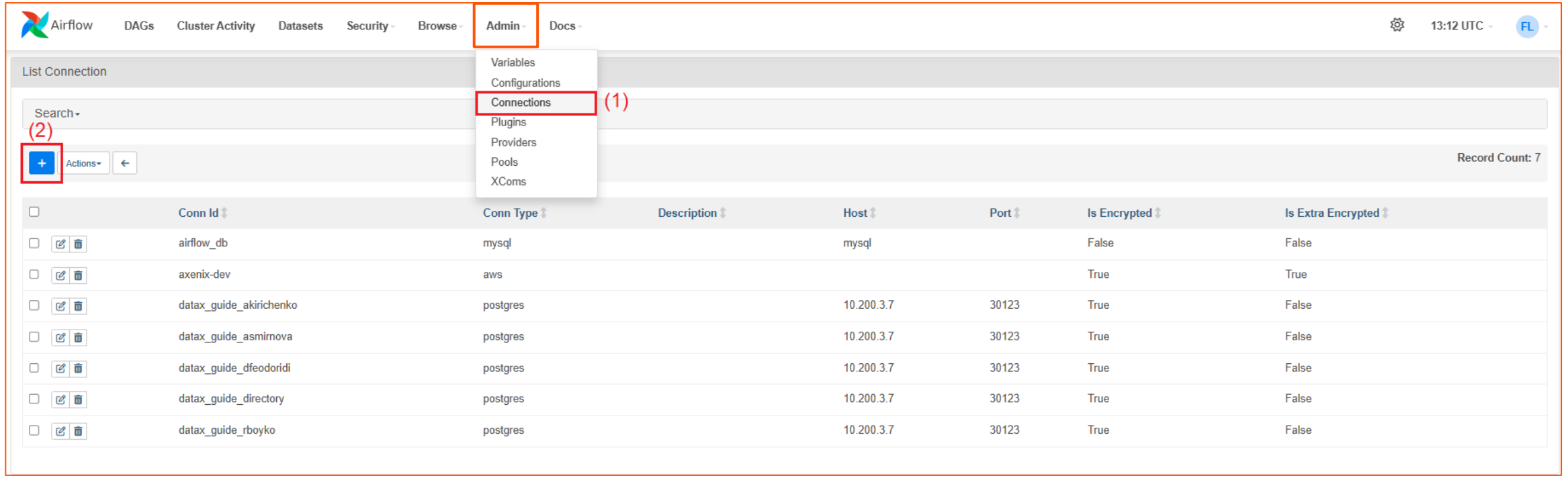
При таком запуске Dat.ax формирует DAG для выбранного пайплайна. Перейти к нему можно через кнопку **DAG последнего запуска (2)** в верхнем меню, после чего откроется интерфейс Airflow с информацией о



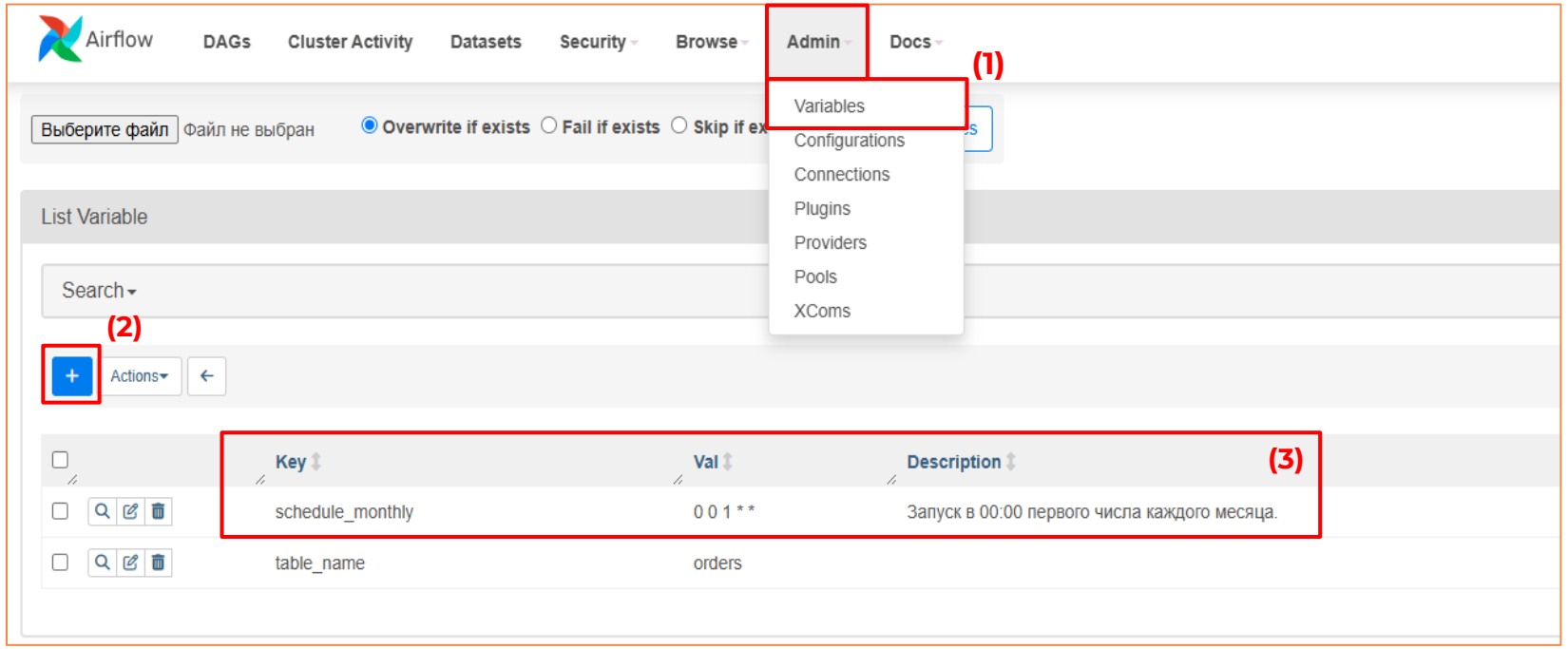
# Запуск в Airflow

**Важное замечание**: Airflow – это внешний сервис для Dat.ax ETL, поэтому подключения настроенные в Dat.ax, не создаются автоматически в Airflow. Это значит, что если ваше подключение используется в Airflow впервые, его необходимо дополнительно создать в разделе Admin → Connections (1), нажав кнопку добавления нового подключения (2). При этом **Connection ID** в Airflow должен соответствовать **Slug** подключения в Dat.ax.



# Расписание

Для запуска пайплайна по расписанию в Dat.ax используется переменная Airflow. Переменная создаётся в разделе Admin → Variables (1) с помощью кнопки добавления новой переменной (2). В значении переменной указывается расписание в формате cron.
Например, для запуска пайплайна в 00:00 первого числа каждого месяца создадим Variable schedule_monthly (3) со значением 0 0 1 * *.



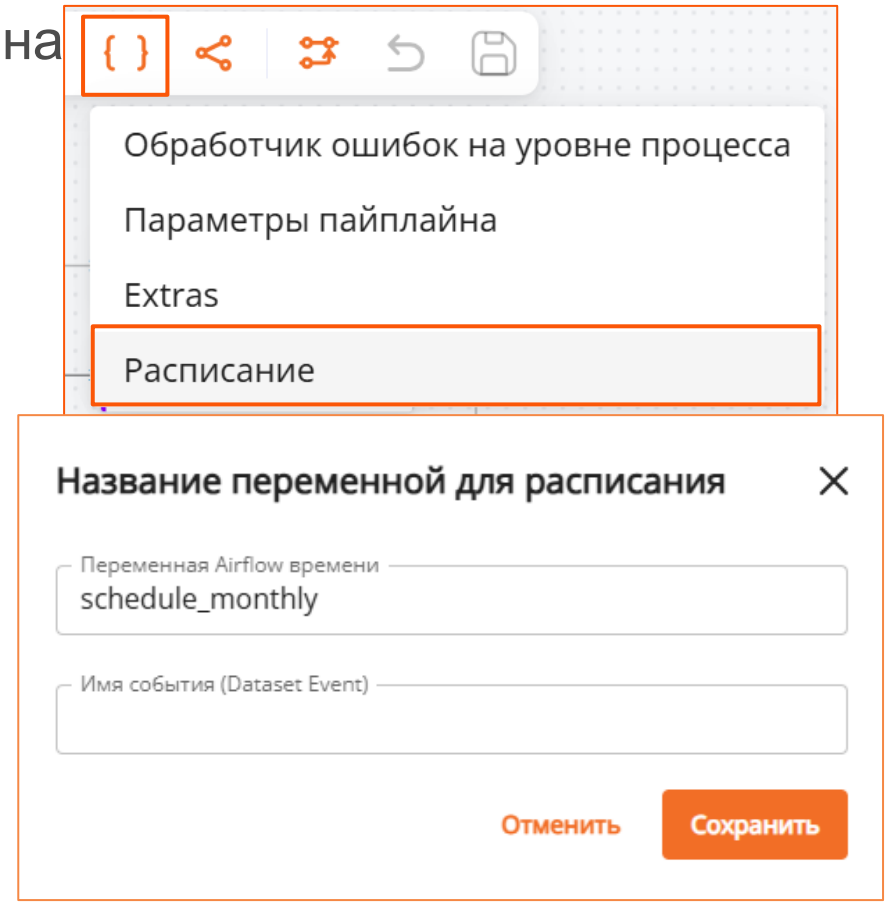

```
0 0 1 * *
| | | | └─ День недели (0-6 или SUN-SAT)
| | | └─── Месяц (1-12 или JAN-DEC)
| | └───── День месяца (1-31)
| └─────── Час (0-23)
└───────── Минута (0-59)

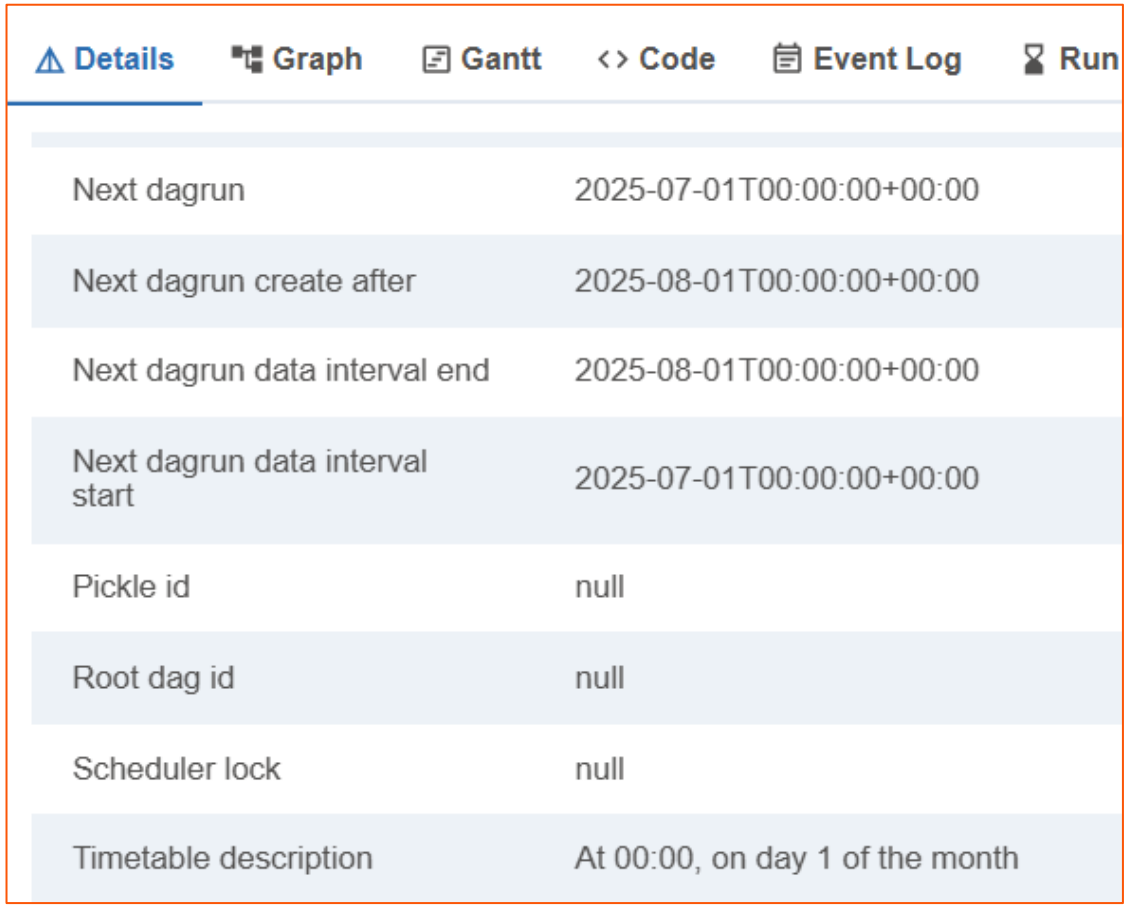
* означает любое допустимое значение поля
```

# Расписание

После создания переменной в Airflow необходимо вернуться в Dat.ax и задать расписание в

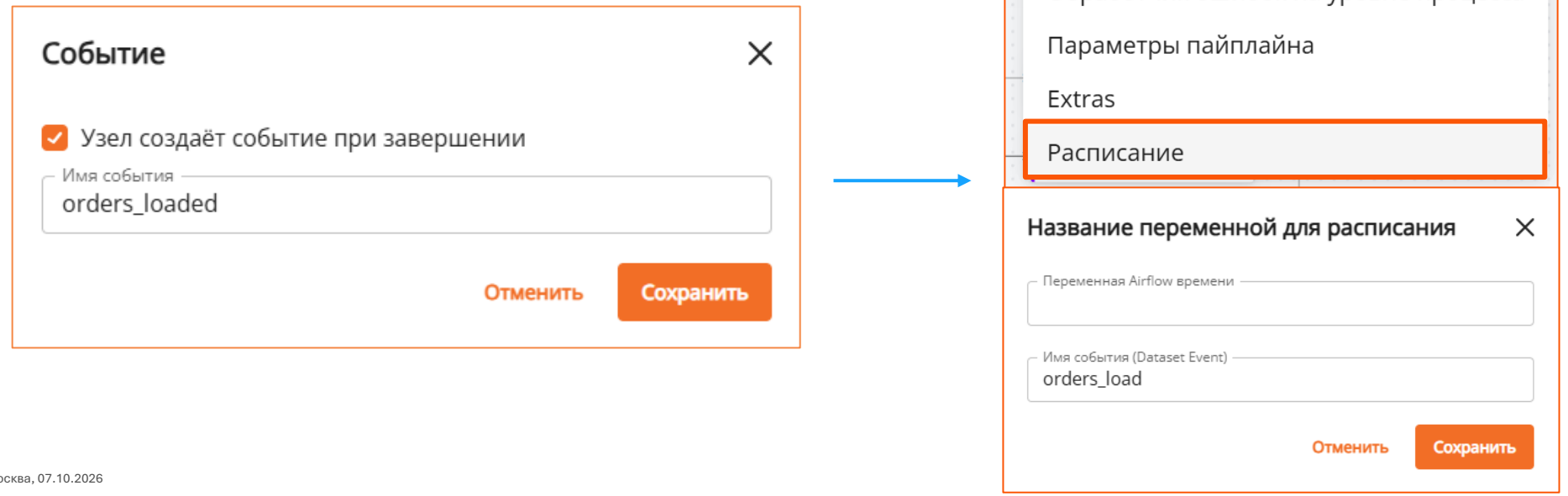
на



# События

В Dat.ax запуск пайплайна по завершению узла другого пайплайна реализован с помощью механизма Datasets в Airflow. Для узла задаётся **Имя события (Dataset Event)**, которое создаётся после его успешного завершения. Это же имя указывается в настройках запуска зависимого пайплайна.
Для настройки события необходимо открыть Действия над узлом → Событие, включить создание события при завершении узла и указать его имя. После сохранения события нужно перейти в пайплайн, который будет запускаться после выполнения этого узла, открыть Настройки запуска → Расписание и указать имя созданного события.

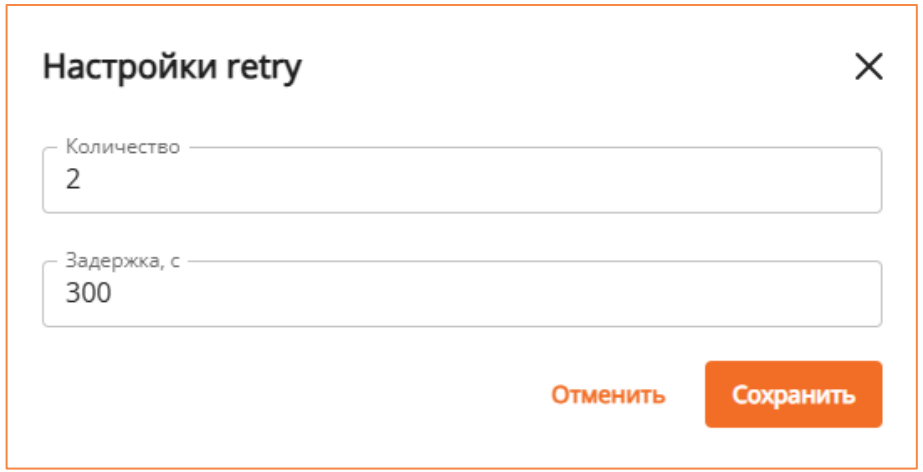


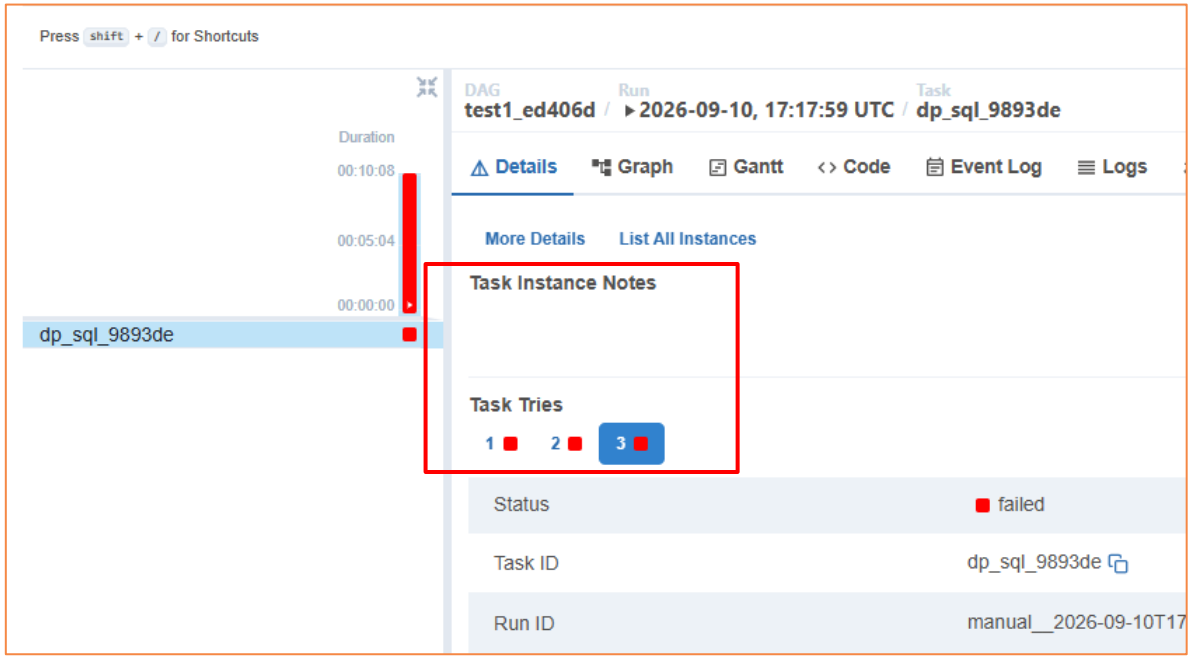
# Повторная попытка выполнения задачи

В Dat.ax для узла можно настроить **retry** – автоматический повторный запуск при возникновении ошибки. Для этого в разделе Действия над узлом → Настройки retry указываются количество повторных запусков и задержка между ними.

В примере для узла настроены две повторные попытки с задержкой 300 секунд между запусками.

Результат выполнения повторных попыток отображается в Airflow.



# Спасибо!

Контакты:

Павел Мельников
Руководитель ИТ-проектов, Axenix
pavel.melnikov@axenix.pro
+79160053455

